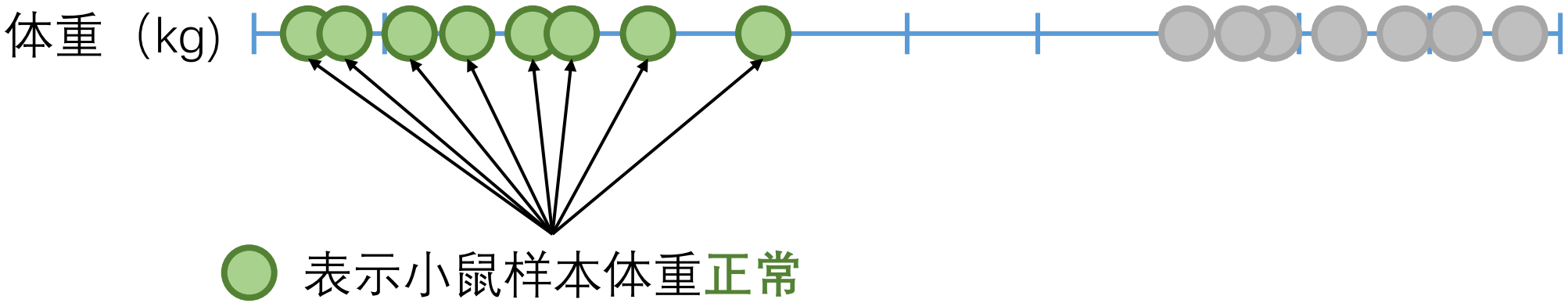
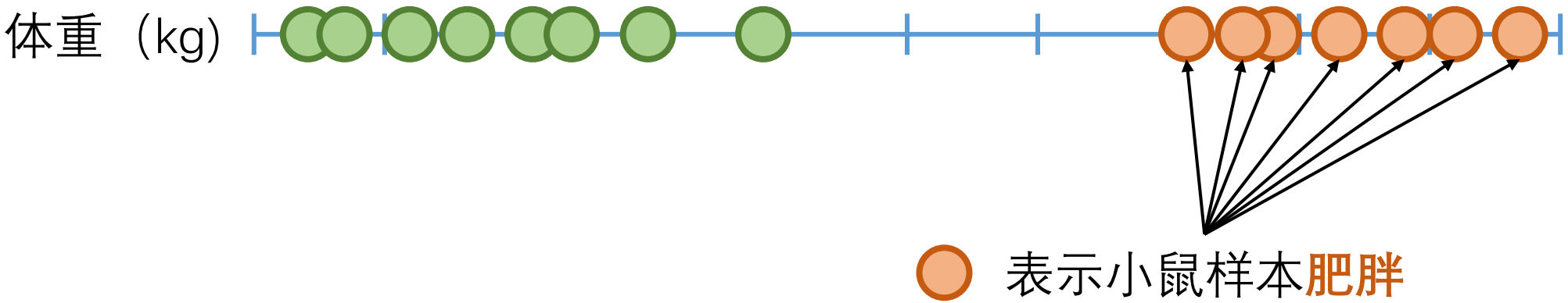
3.2 常用的分类模型—

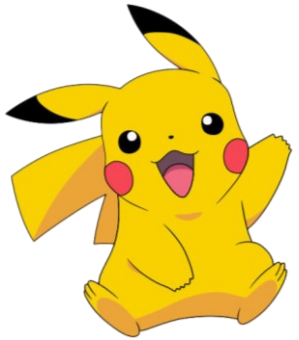
支持向量机

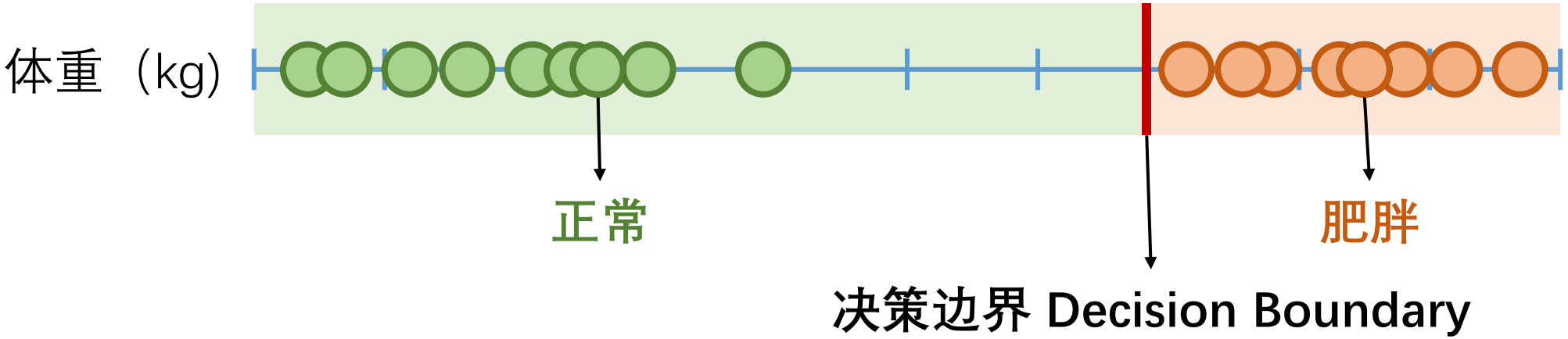
Support Vector Machine

体重（kg）

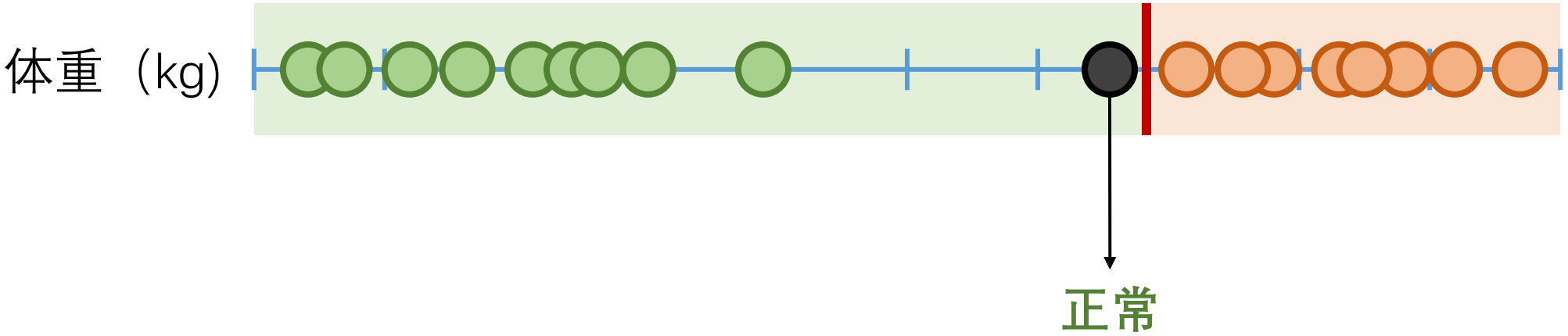

体重（kg）
表示小鼠样本体重正常

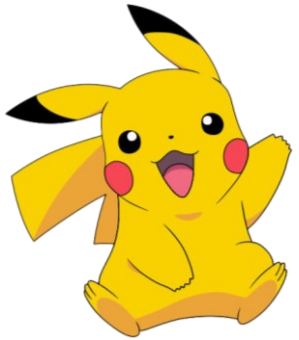

体重（kg）
表示小鼠样本肥胖

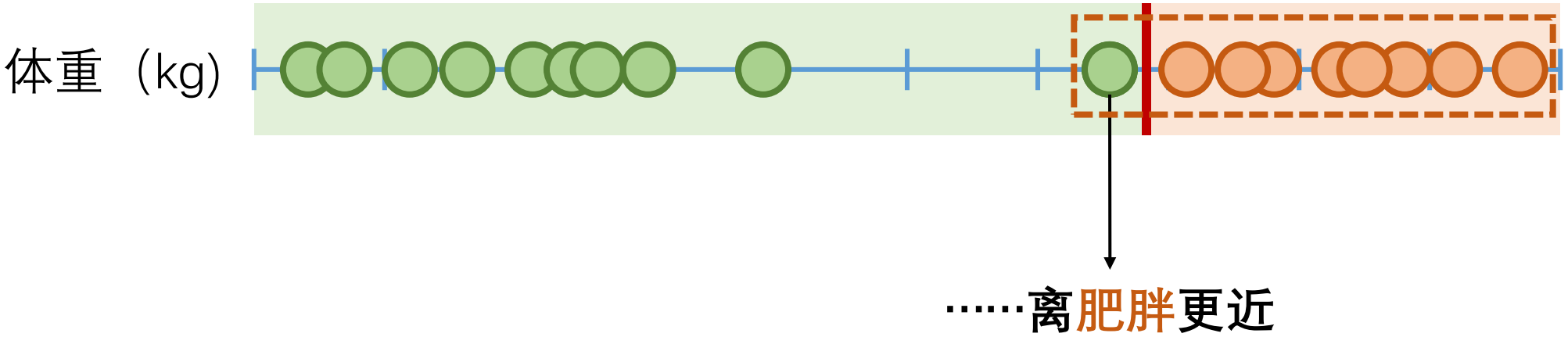


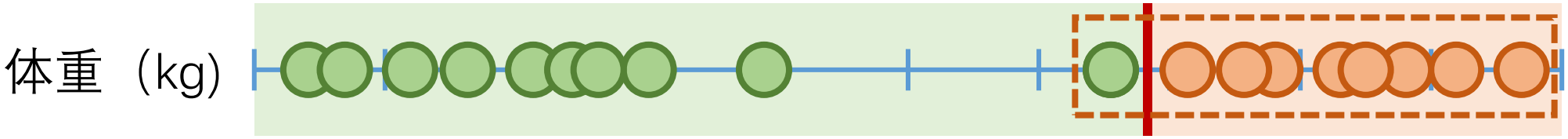


根据决策边界可以判断是否肥胖


体重（kg）
正常



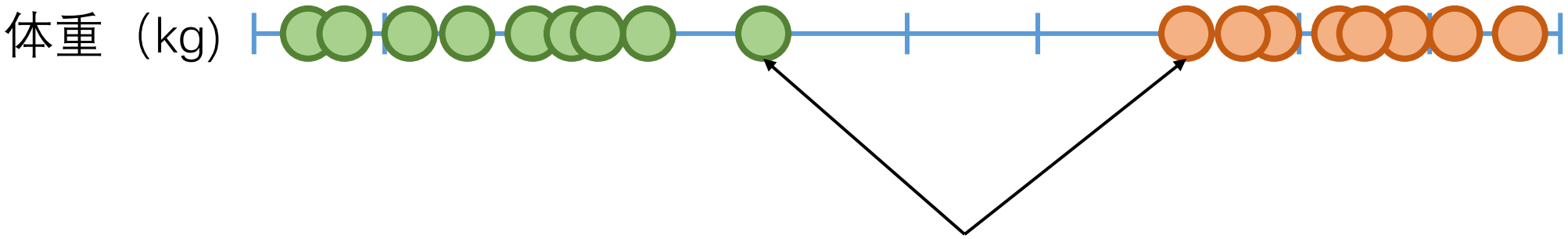

体重（kg）
……离肥胖更近



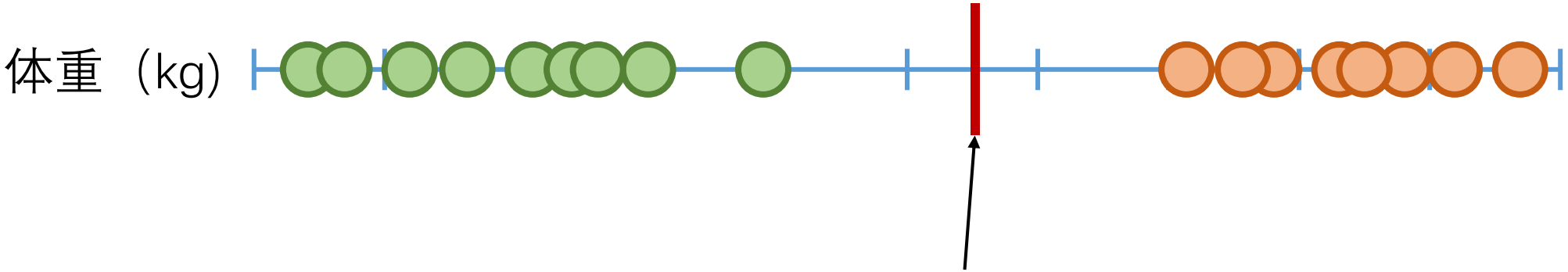
这样随意的设定决策边界存在缺陷

那么有更好的方法吗?

必须有！

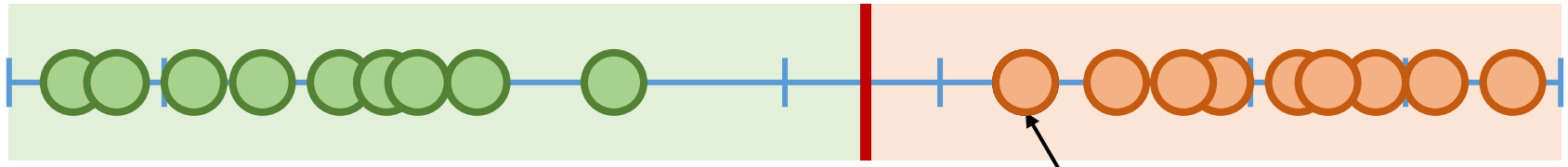


我们可以关注两个类别的**边缘样本**

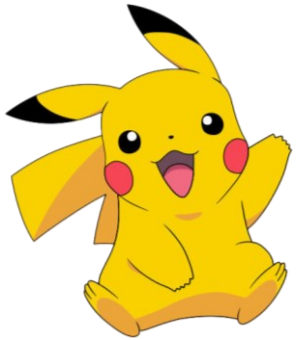


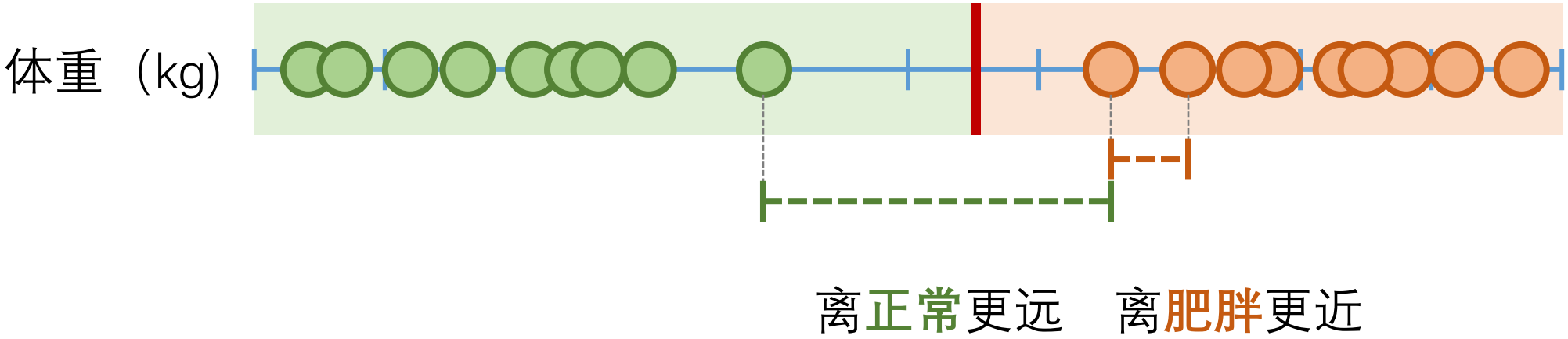
将决策边界定在**边缘样本中间**

体重（kg）

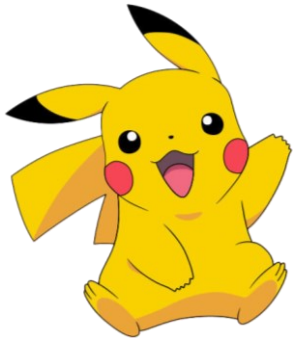


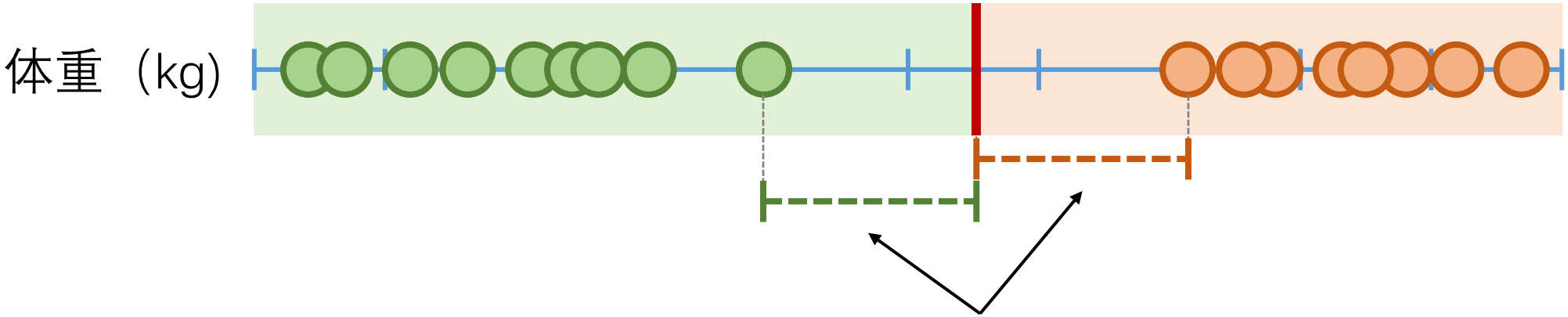
回到之前的例子 肥胖





术语警告

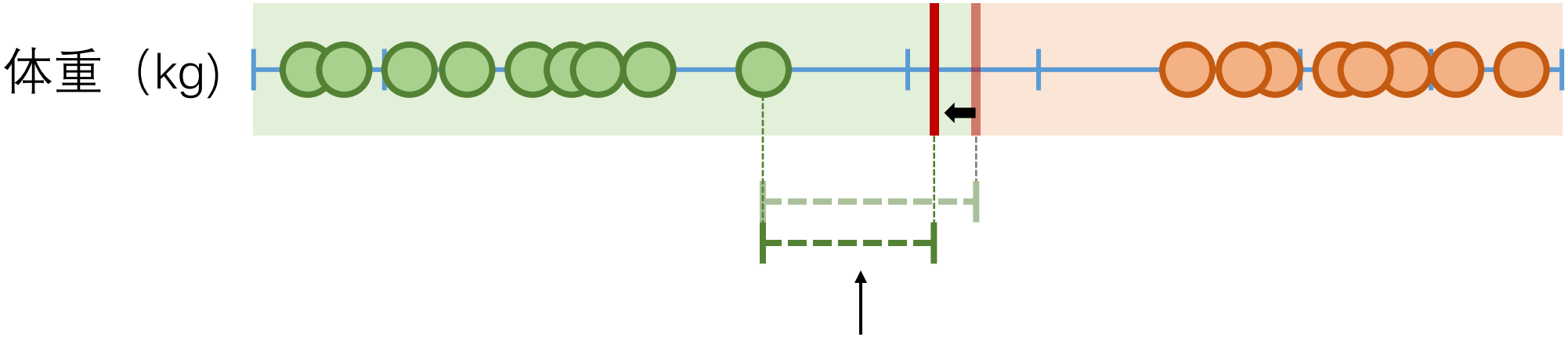




分类间隔（Margin）

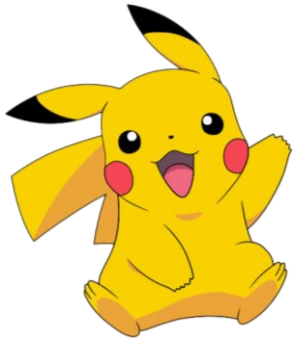
边缘样本和决策边界之间的**最短**距离

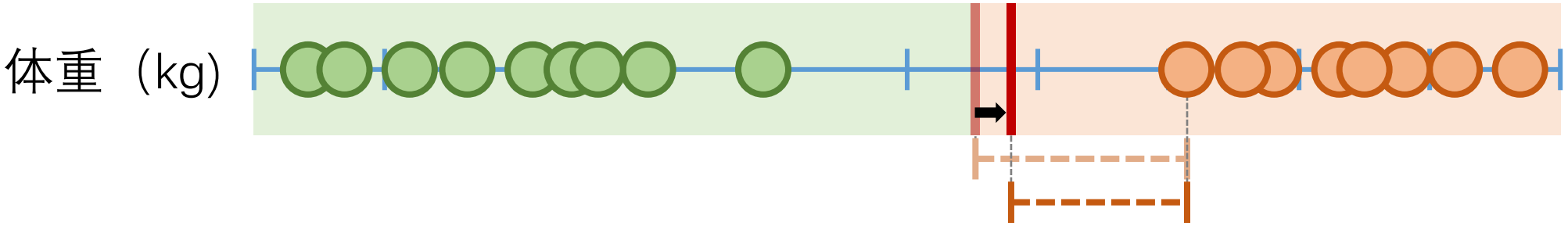
当决策边界处于中间位置时，分类间隔最大



假如决策边界向**正常**类别靠近

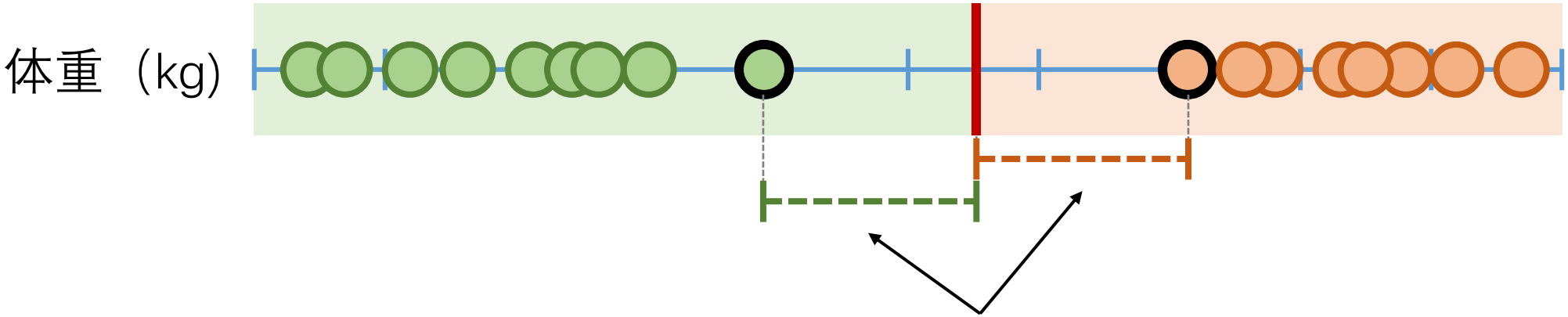
距离缩短分类间隔缩小





假如决策边界向**肥胖**类别靠近

距离缩短分类间隔缩小

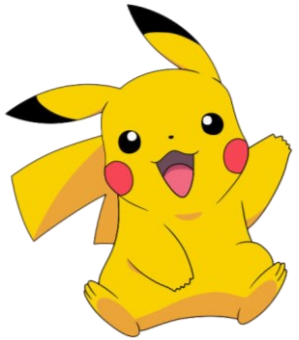


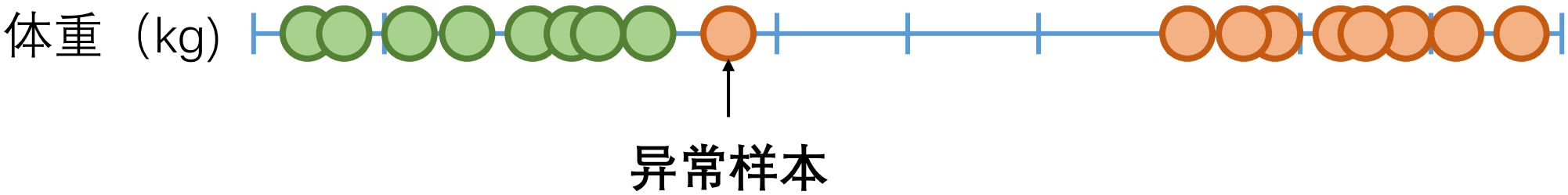
当以最大的分类间隔作为分类依据

最大边际分类器

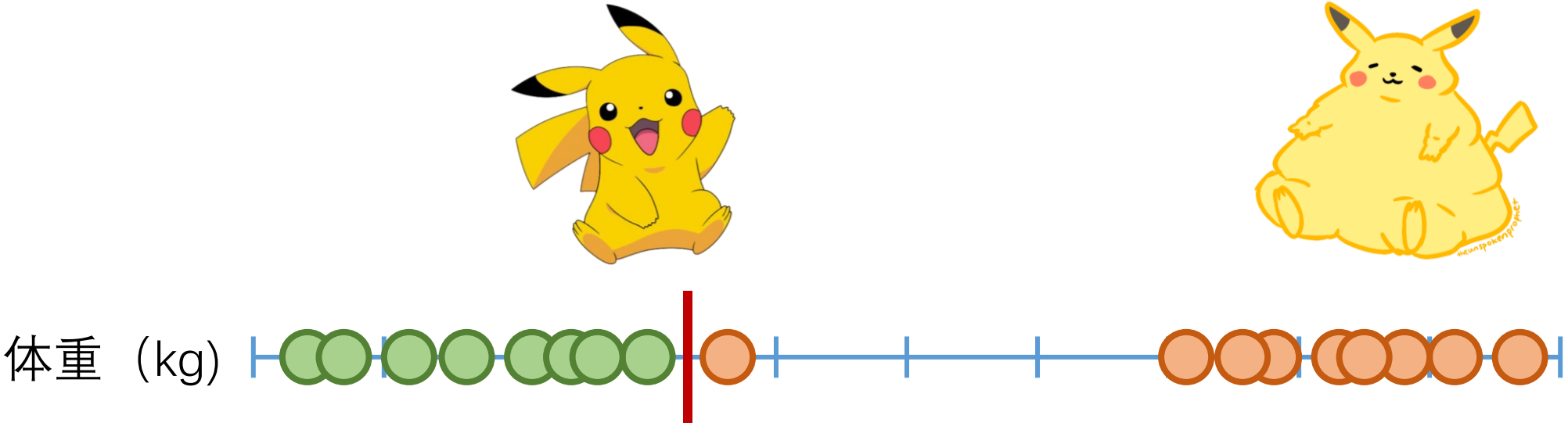
Maximal Margin Classifier

好像效果很不错的样子



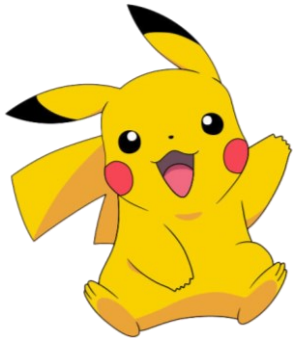


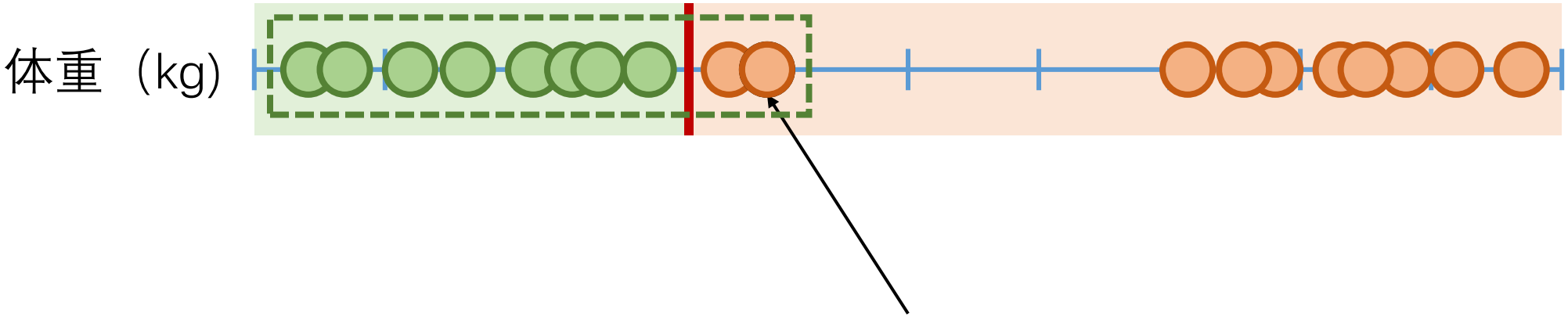
假如训练数据是这样的……



最大边际分类器过于接近**正常**样本

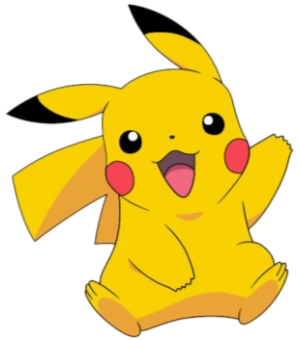
距离大部分**肥胖**样本过远

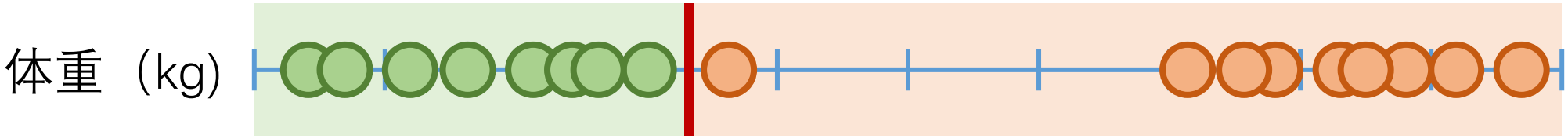




这个新样本将被归类为**肥胖**

实际上该样本为**正常**的可能性更大

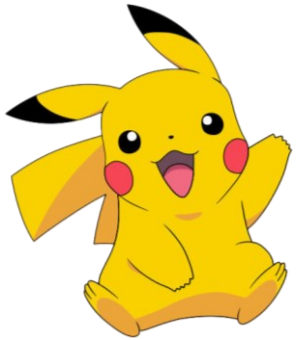




观察：**最大边际分类器**对于异常值非常敏感

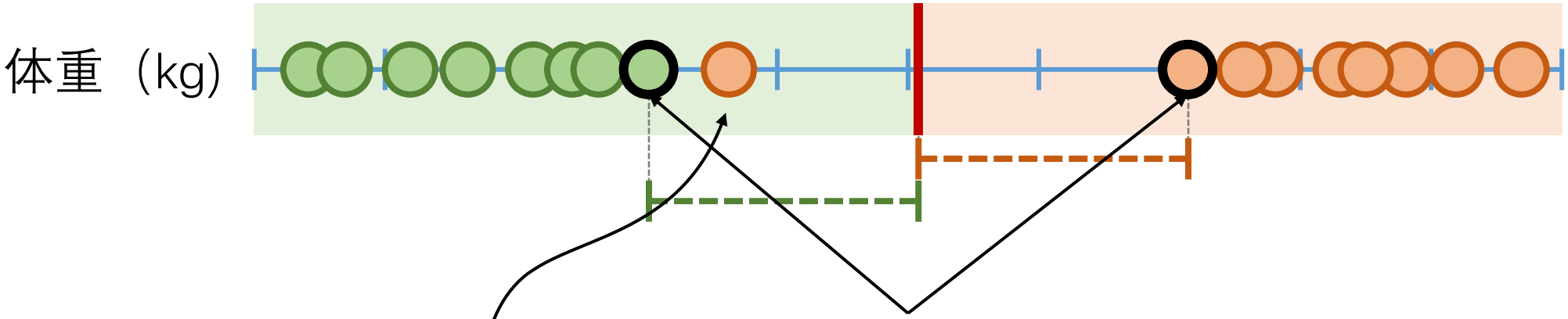
异常值可能极大降低分类效果

如何改进?



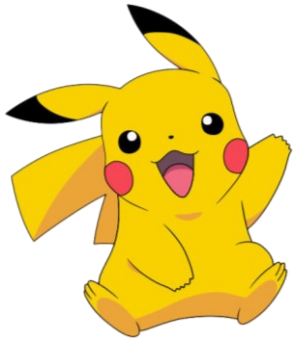
为了降低异常值带来的影响，可以……

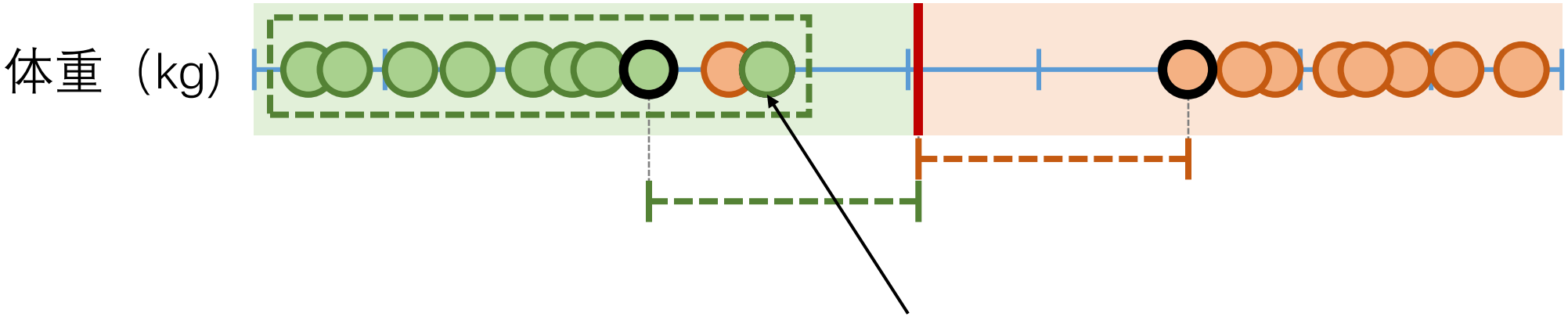
允许错误分类的结果



以这两个点构建最大边际分类器

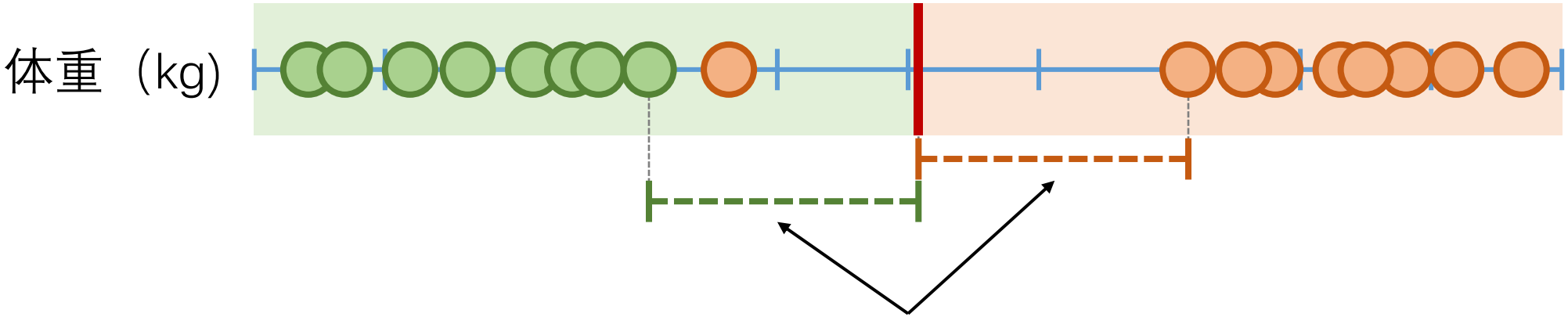
该**肥胖**样本被错误地归为**正常**





此时，新样本却可以被正确分类

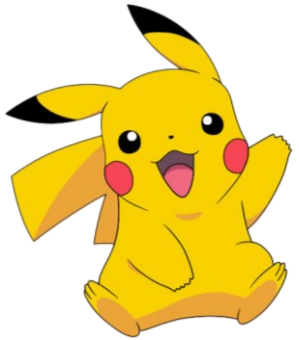
因为新样本距**正常**类别更近

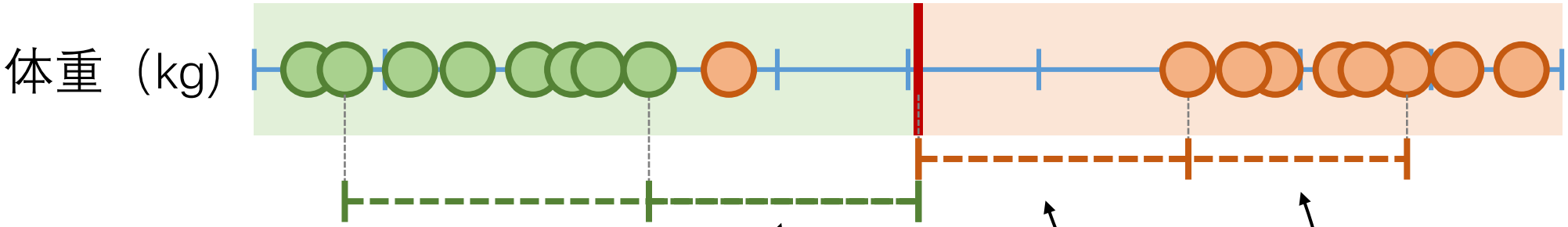


当允许误分类时

“边缘”样本和决策边界之间的最短距离为

软分类间隔 （Soft Margin）

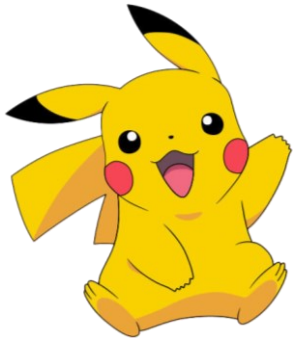


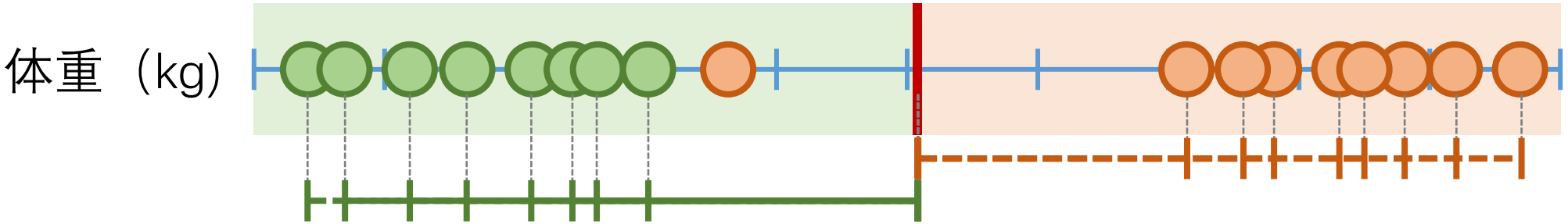


那么问题来了：

是这样的间隔更好？

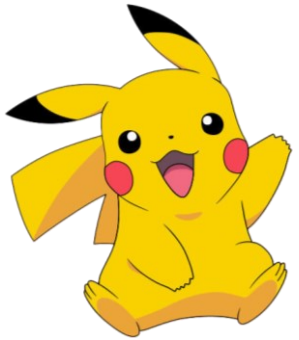
还是这样的间隔更好？

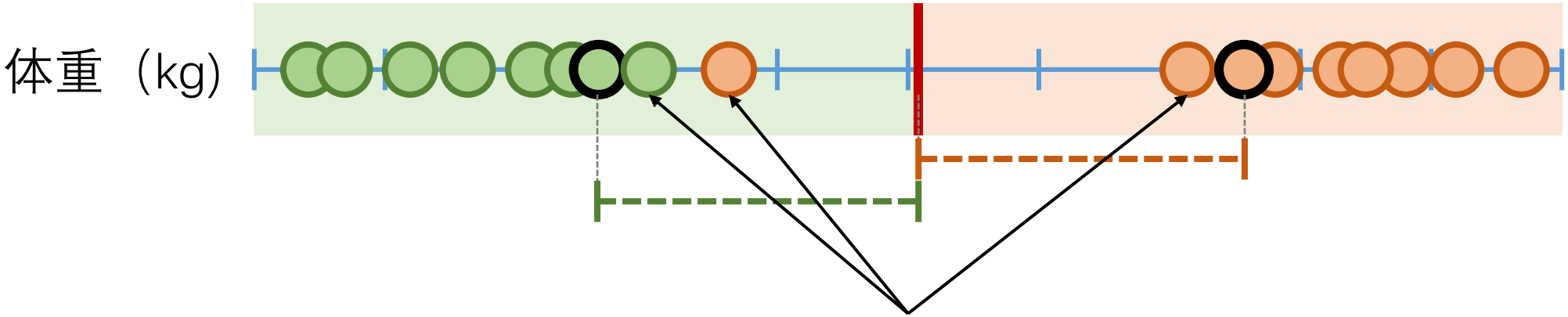




解决的方式：交叉验证

尝试不同的分类间距，以得到最优解

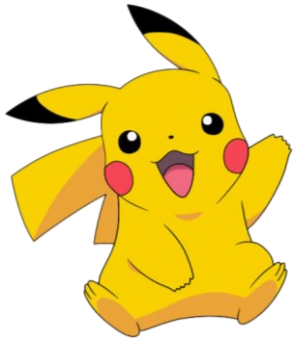


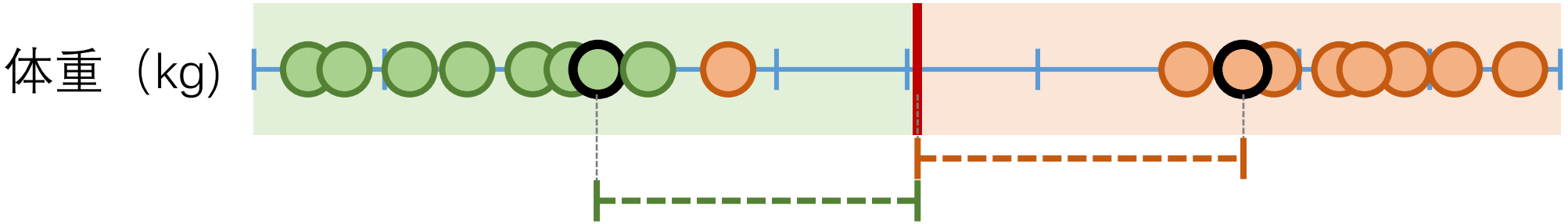


例，当此时为最优解

在软分类间隔之中有：

1 个错误分类　　2 个正确分类



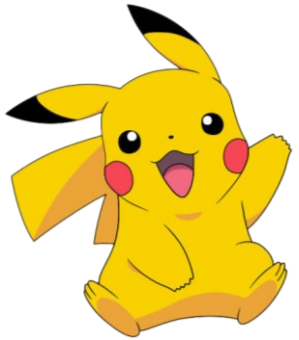


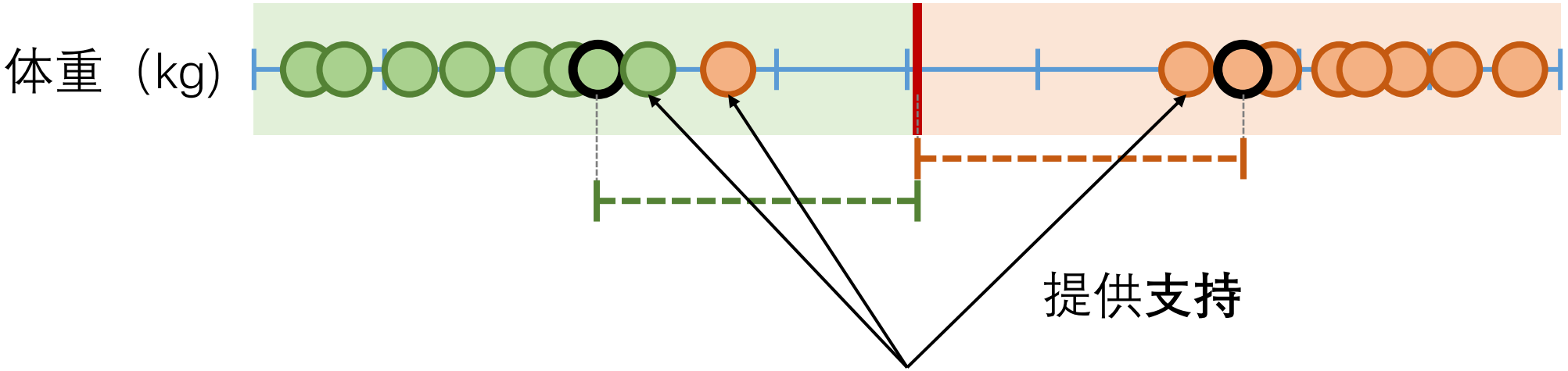
【注意】当使用软分类间隔确定决策边界时

即是说我们使用**软边际分类器**进行分类

支持向量分类器

Support Vector Classifier

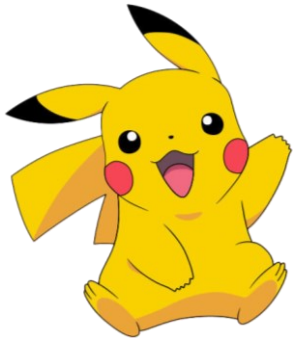


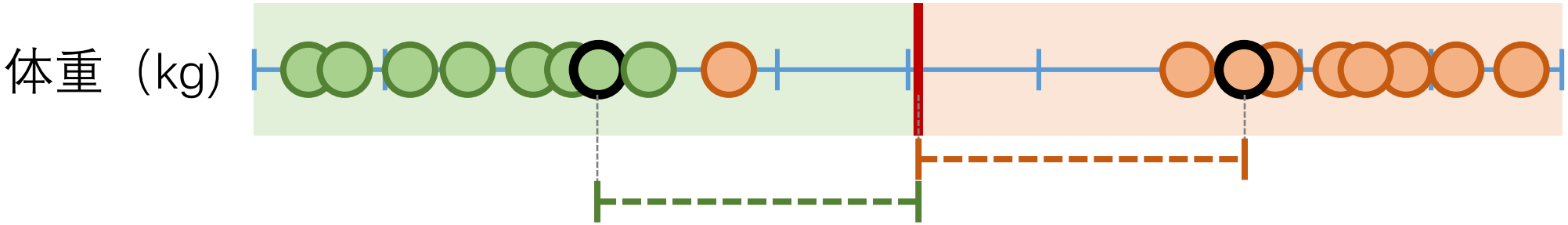


软分类边界内的样本

支持向量分类器

Support Vector Classifier

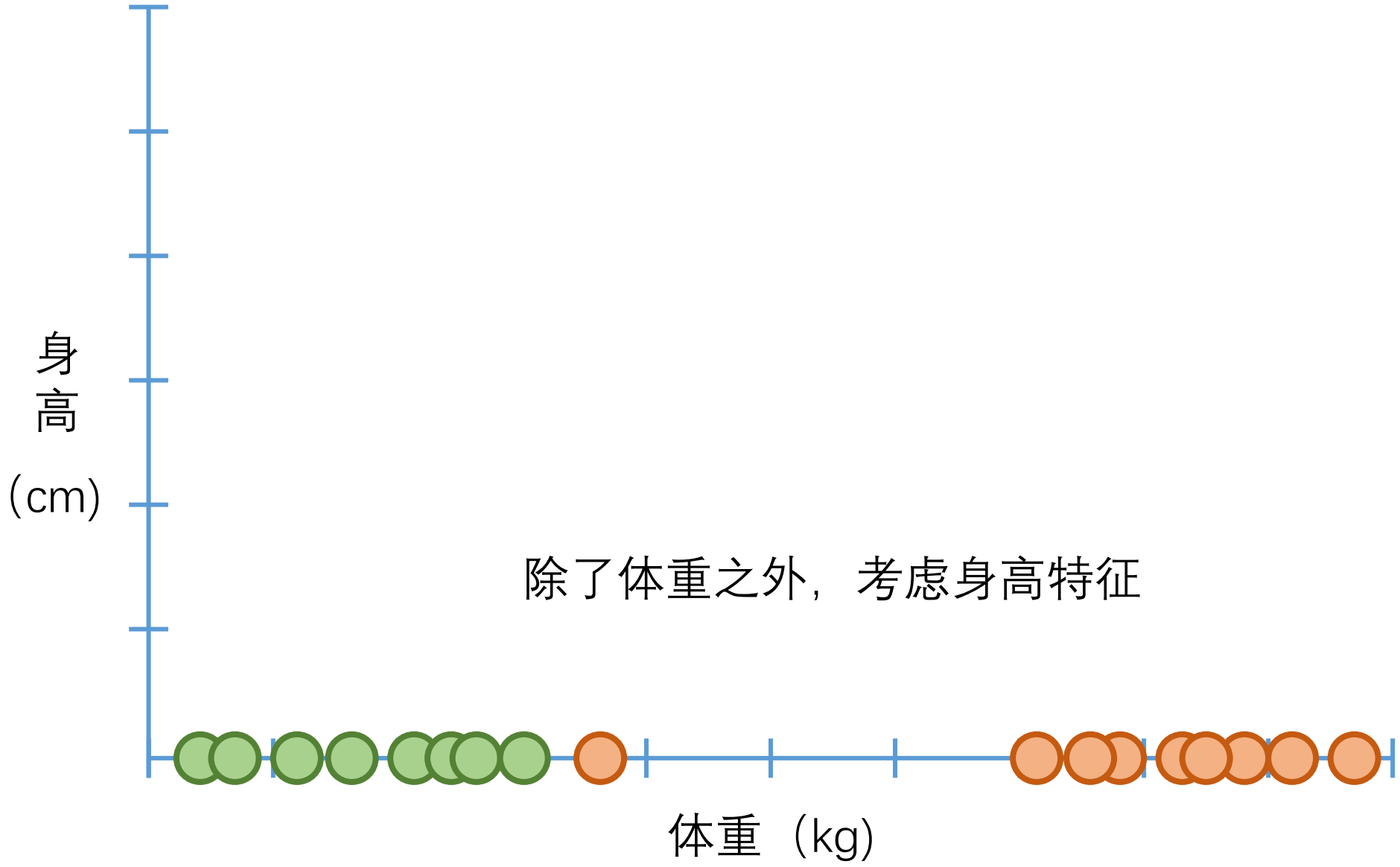




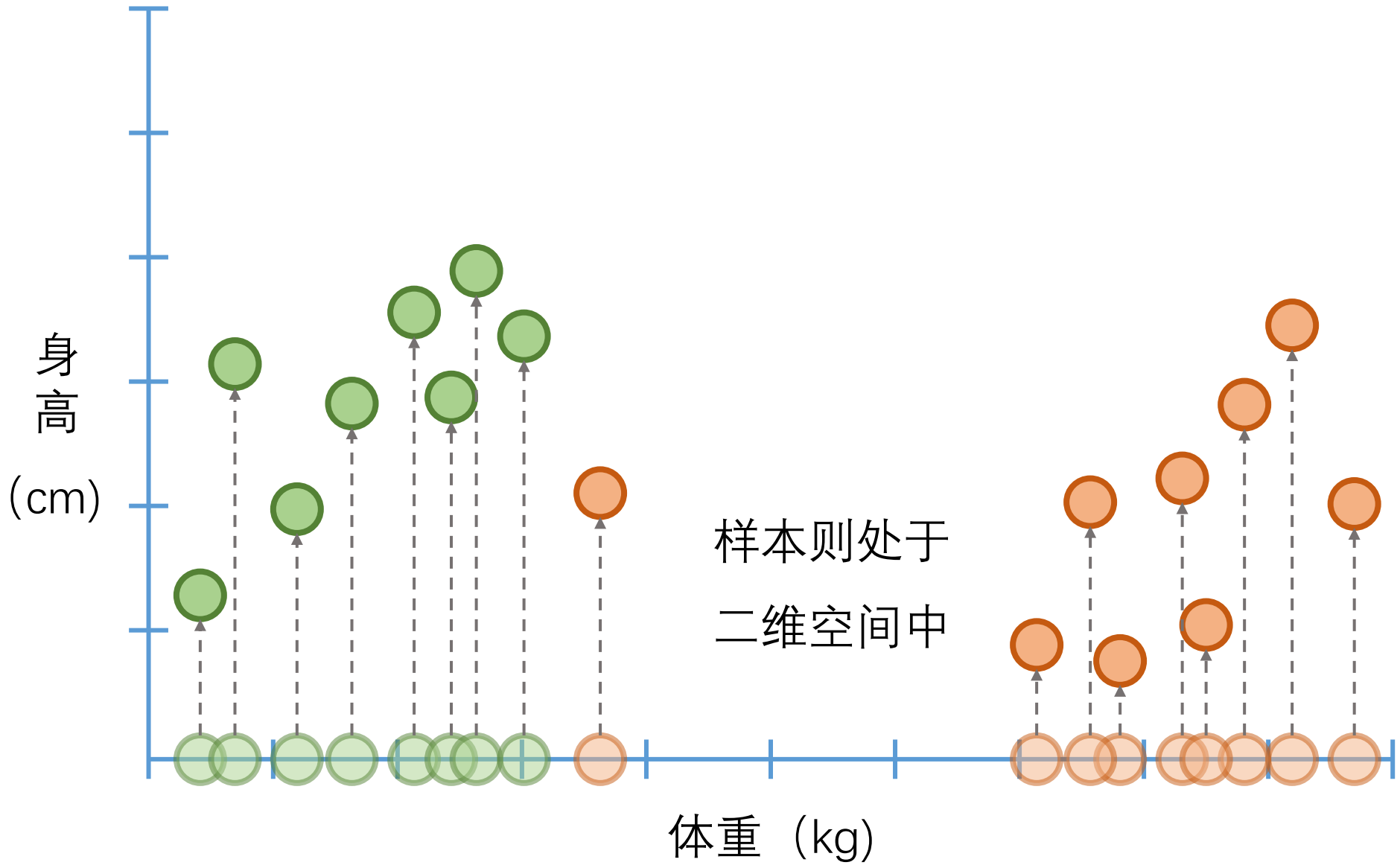
向量指的是什么呢?

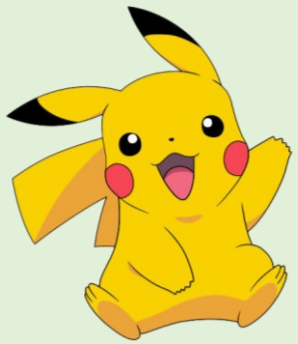
支持向量分类器

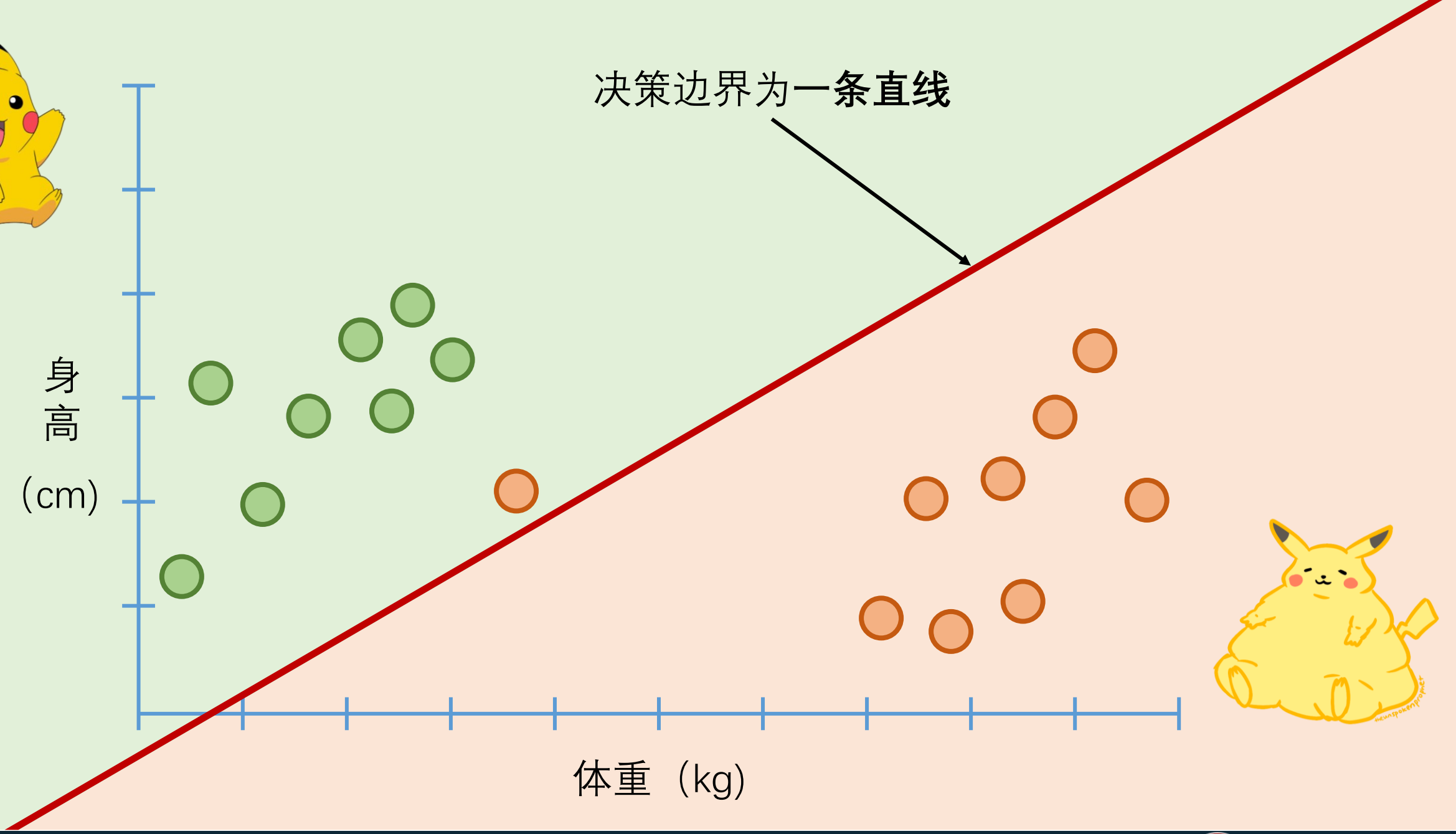
Support Vector Classifier



除了体重之外，考虑身高特征


身
高
(cm)
样本则处于
二维空间中
体重（kg)

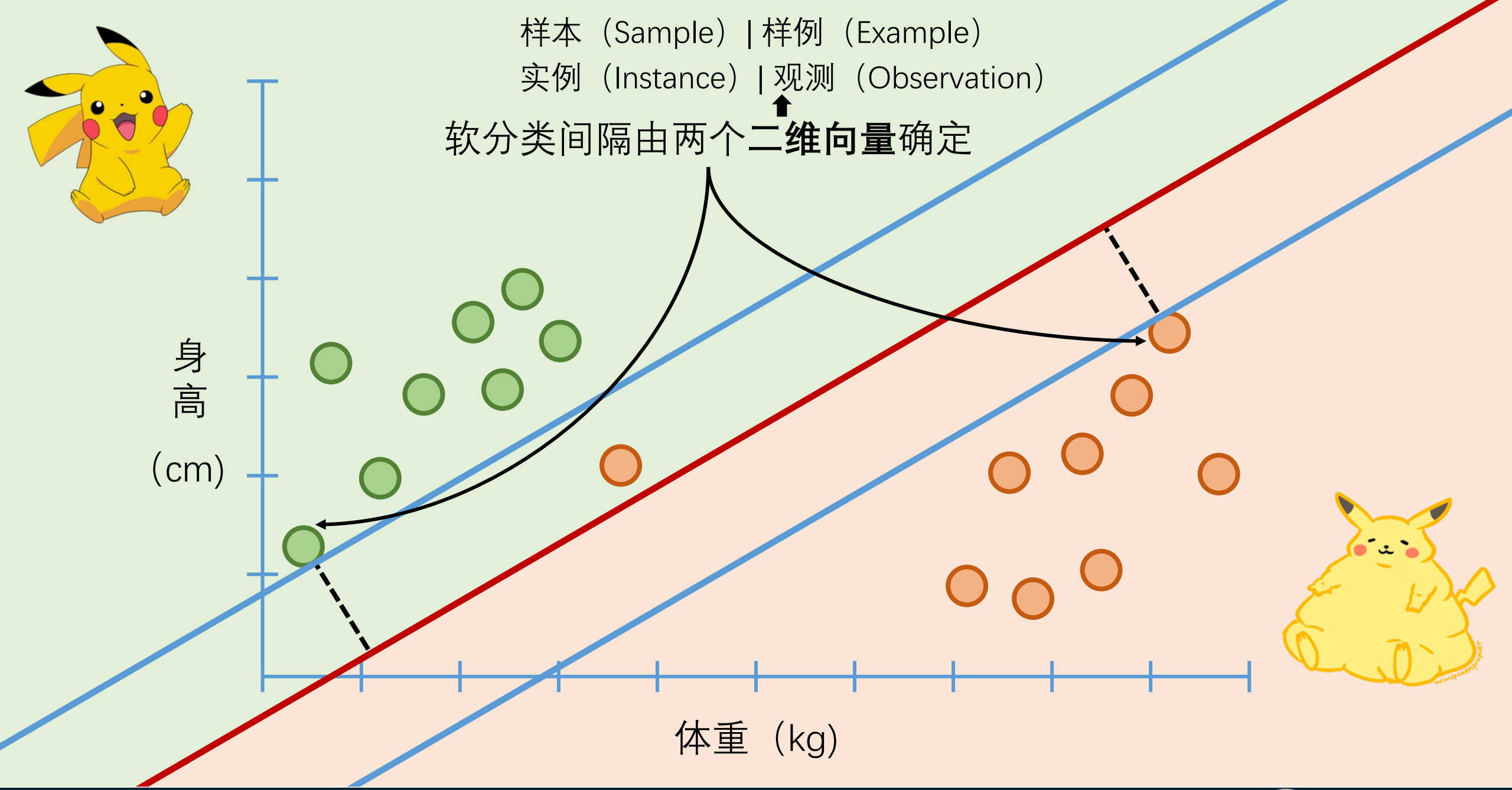



决策边界为一条直线
身高
(cm)
体重（kg)

样本（Sample）| 样例（Example）

实例（Instance）| 观测（Observation）

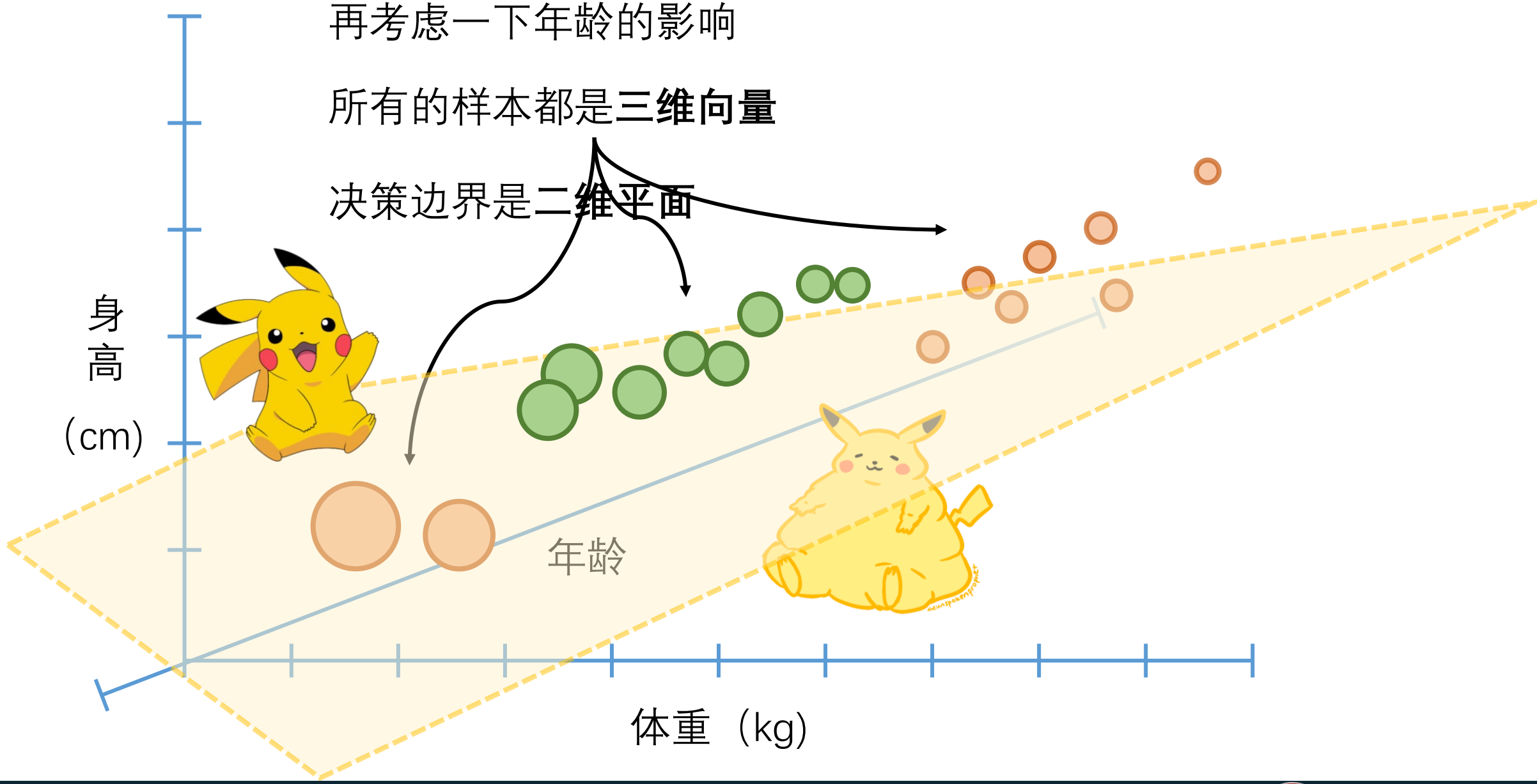
软分类间隔由两个**二维向量**确定



再考虑一下年龄的影响

所有的样本都是**三维向量**

决策边界是**二维平面**

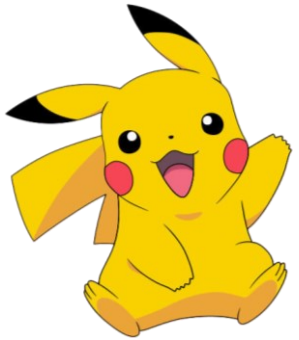


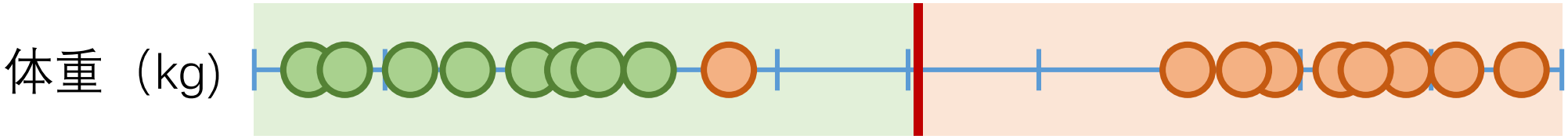
总结一下：支持向量分类器的主要思想

根据软分类边界中的支持向量确定

“**超平面（Hyper Plane）**”

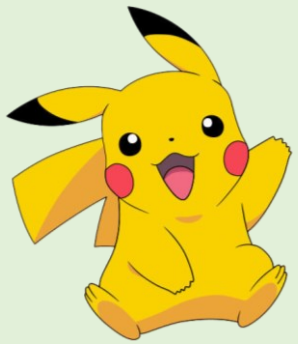
由超平面判断具体的类别





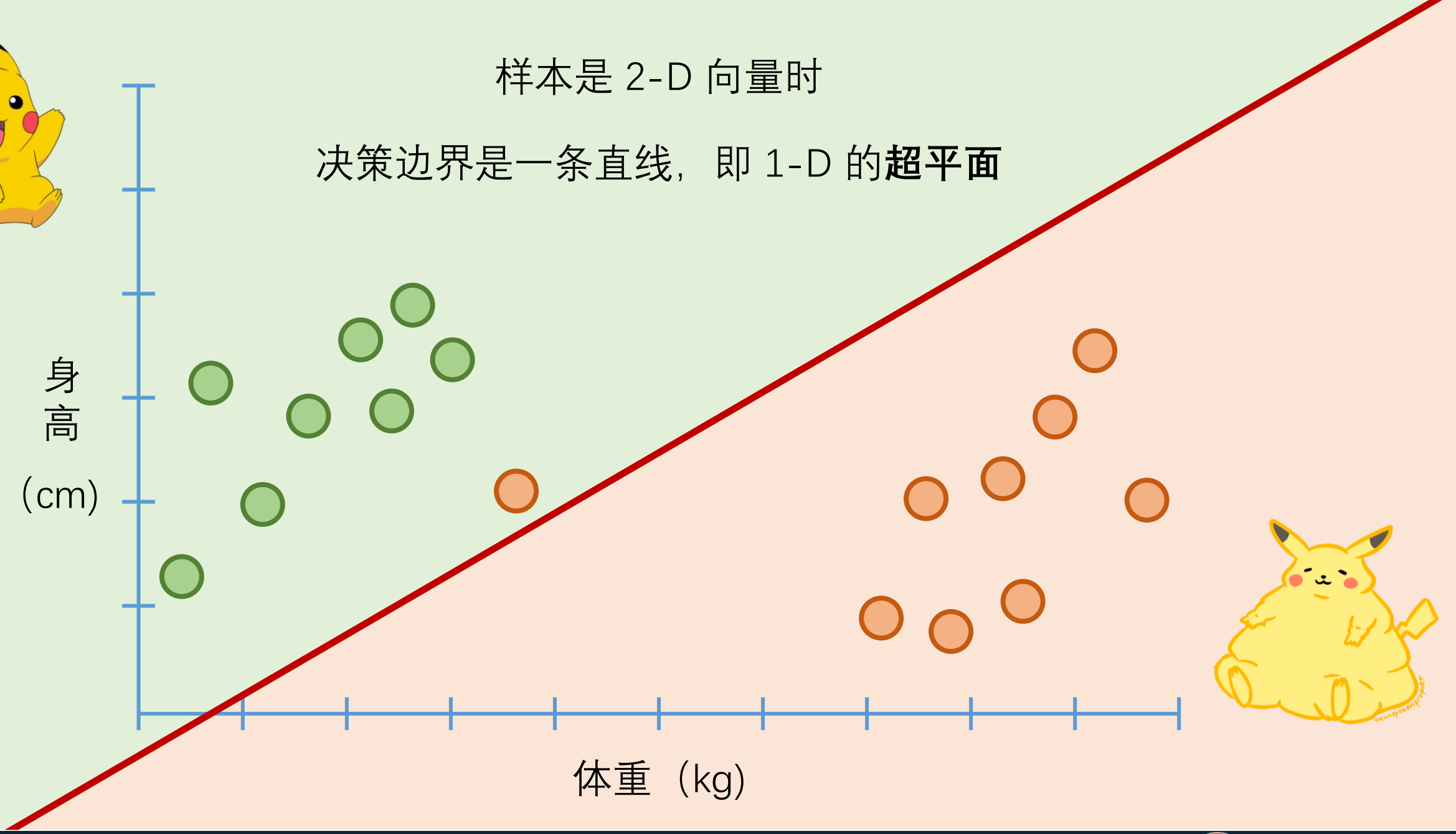
样本是 1-D 向量时

决策边界是一个点，即 0-D 的**超平面**



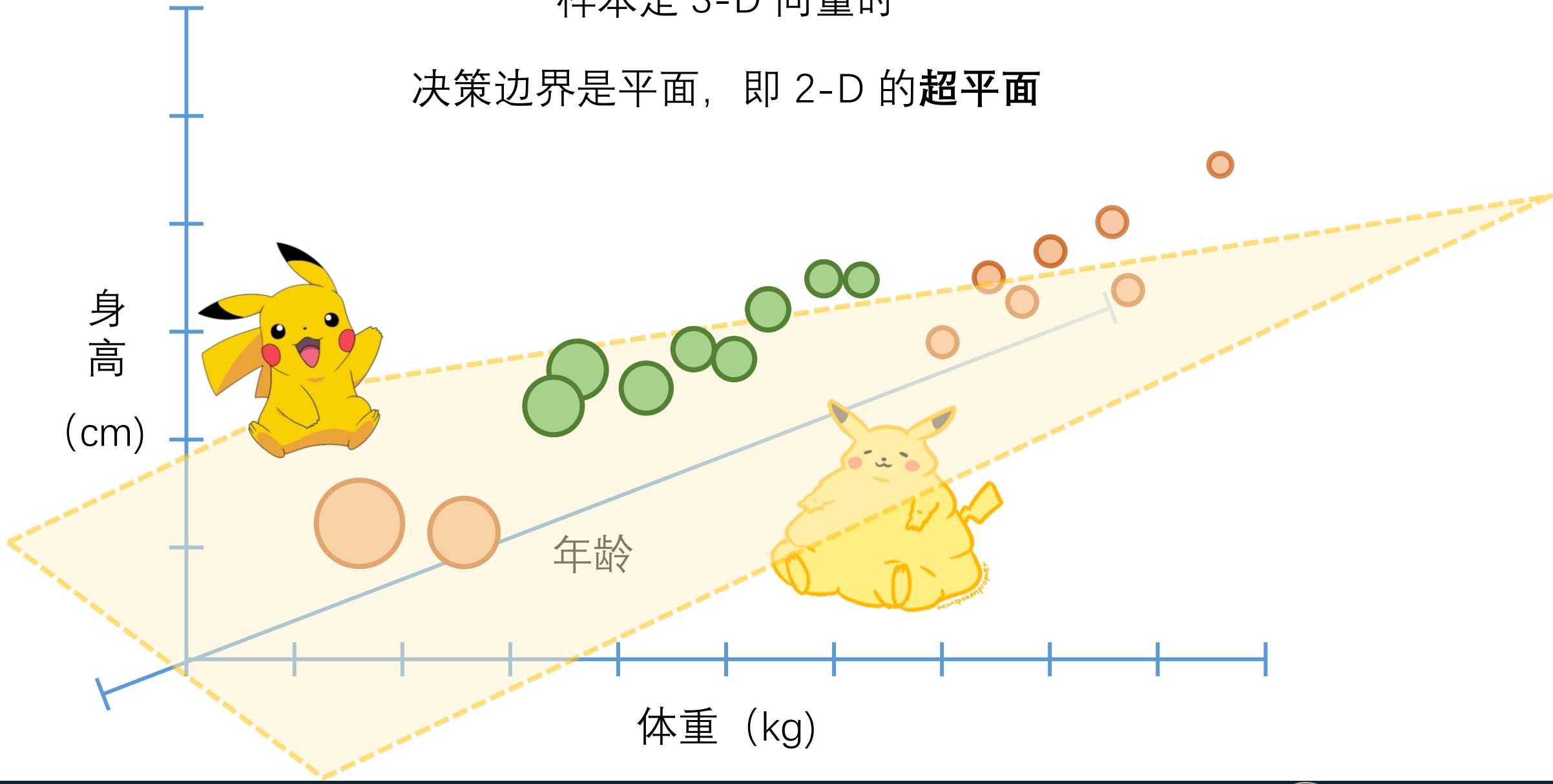
样本是 2-D 向量时

决策边界是一条直线，即 1-D 的**超平面**



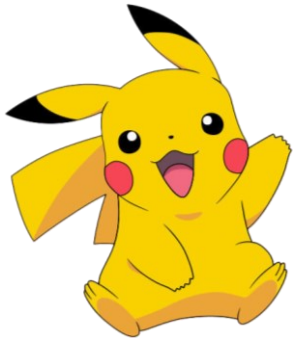
样本是 3-D 向量时

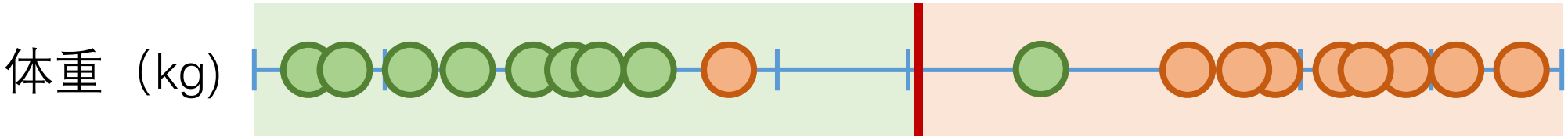
决策边界是平面，即 2-D 的**超平面**



样本是 N 维向量时

决策边界 N-1 维的**超平面**



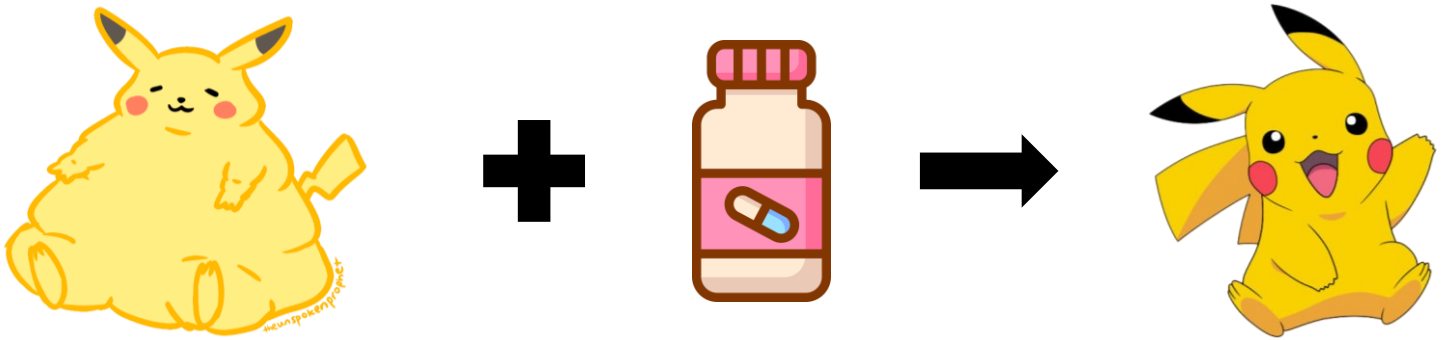


目前为止**支持向量分类器**感觉还不错

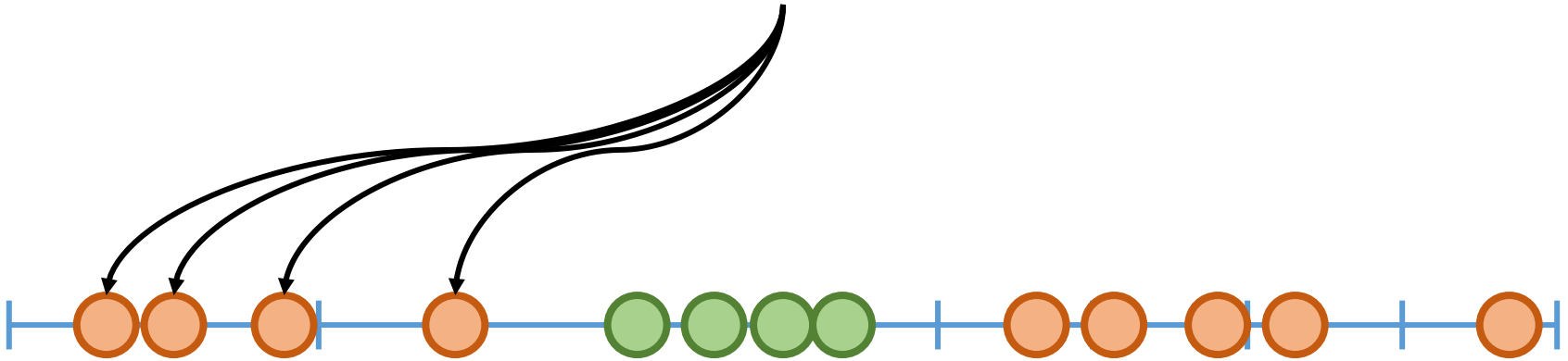
对于训练数据的体量需求不高

能够应对有异常值的情况

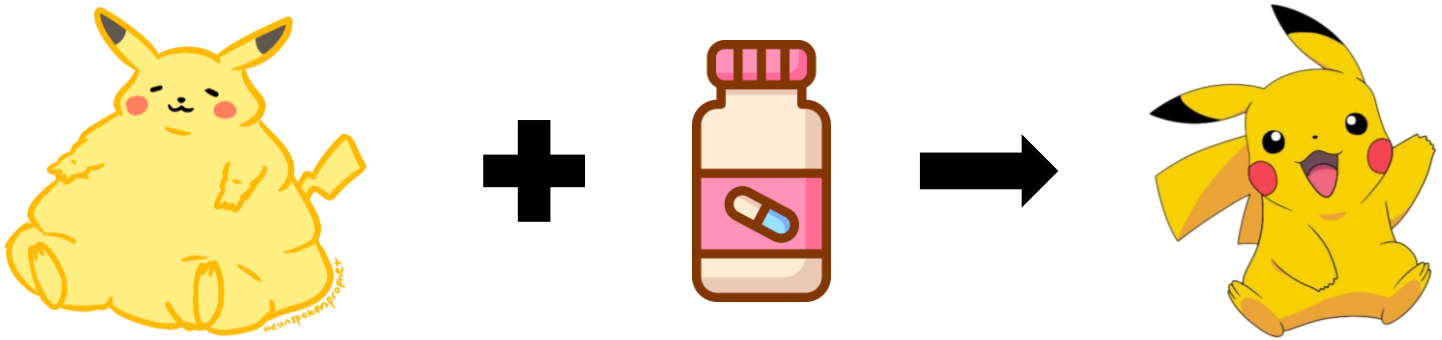
能够应对类别重叠的情况



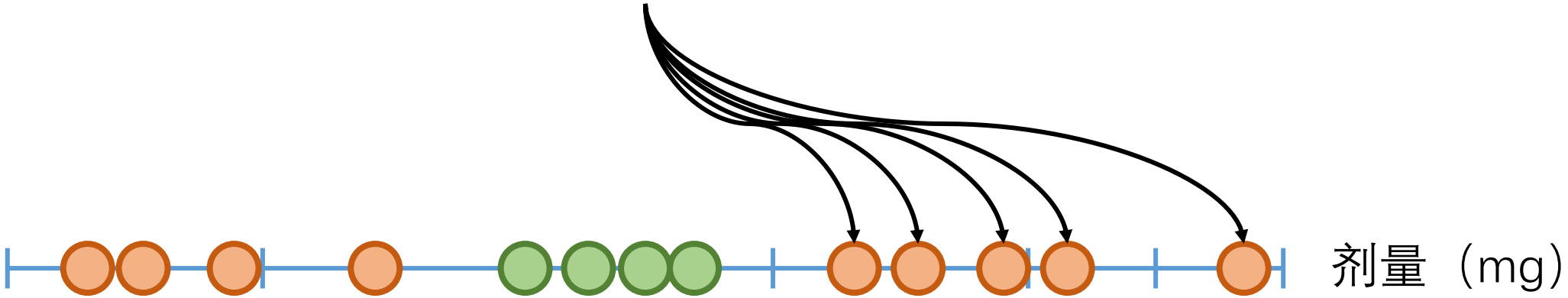
表示**治不好**　　计量太少，没有作用

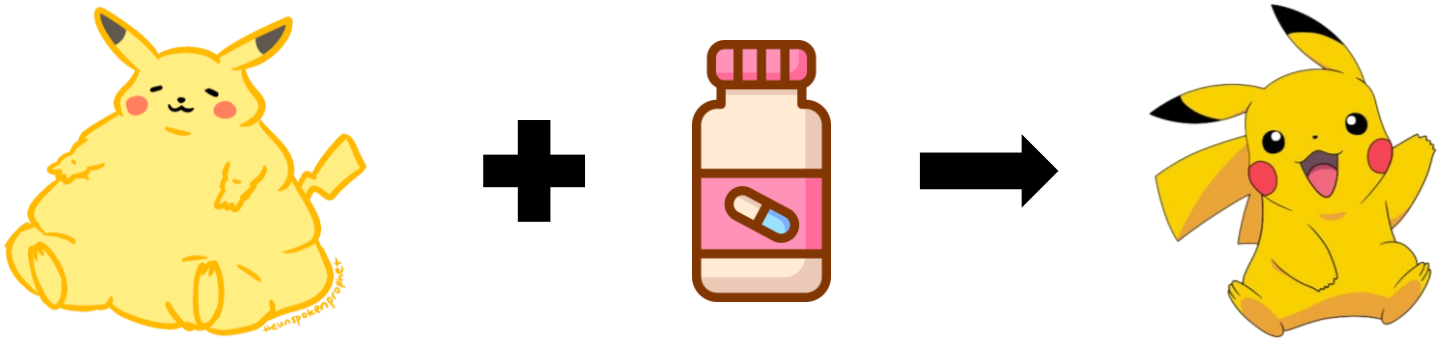


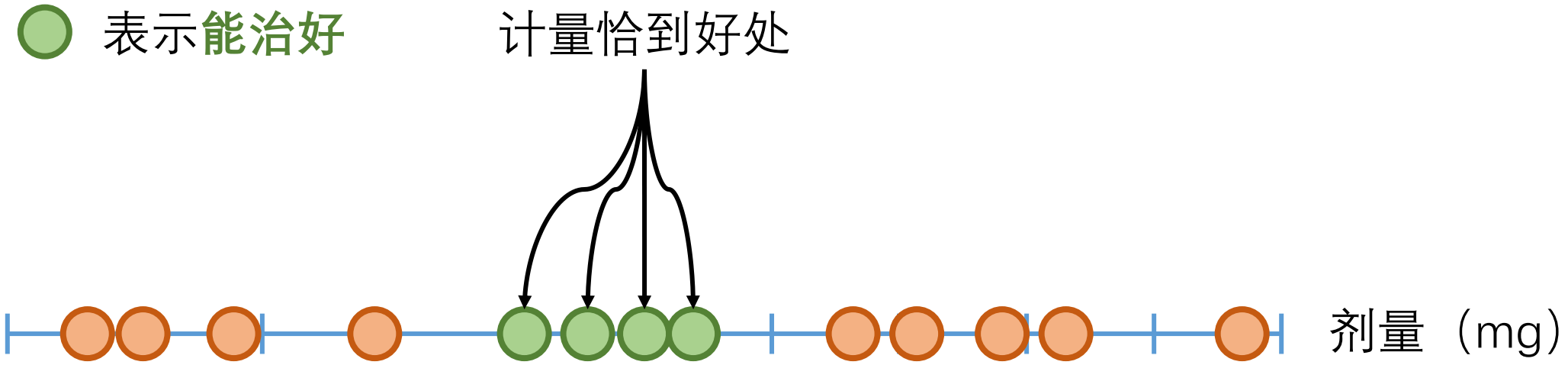
剂量（mg）

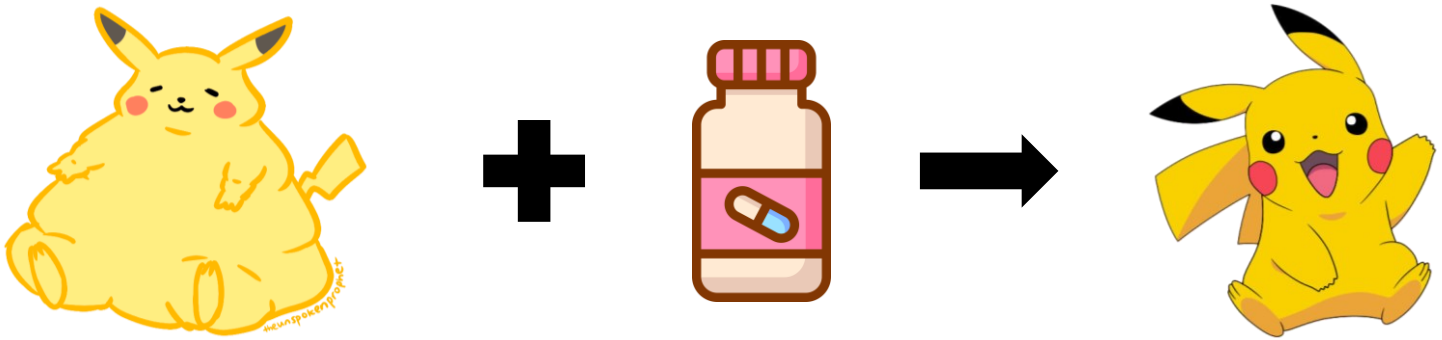


表示**治不好**　　计量太多，GG了





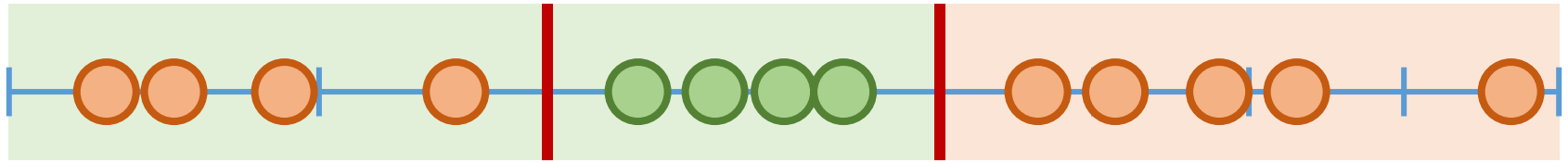

表示能治好
计量恰到好处
剂量（mg）



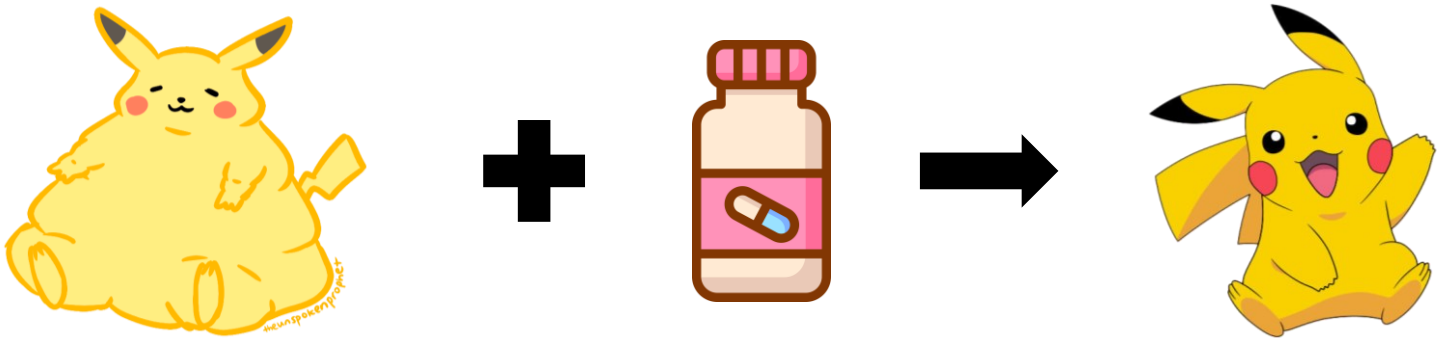
感觉使用**支持向量分类器**无法有效地进行判断

无论怎样设置**软分类间隔**和**决策边界**

都会存在大量的误判

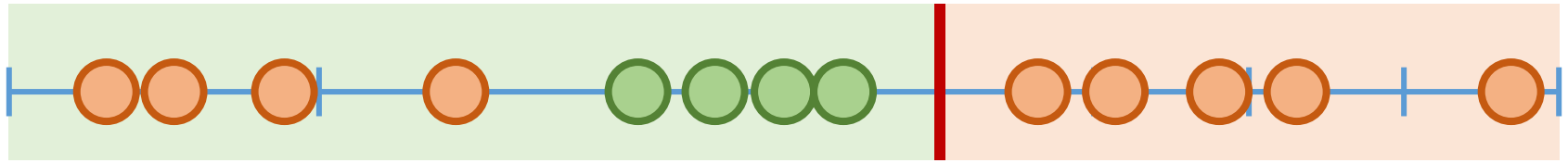


剂量（mg）



对于**线性不可分**的情况，还有比**最大边际分类器**

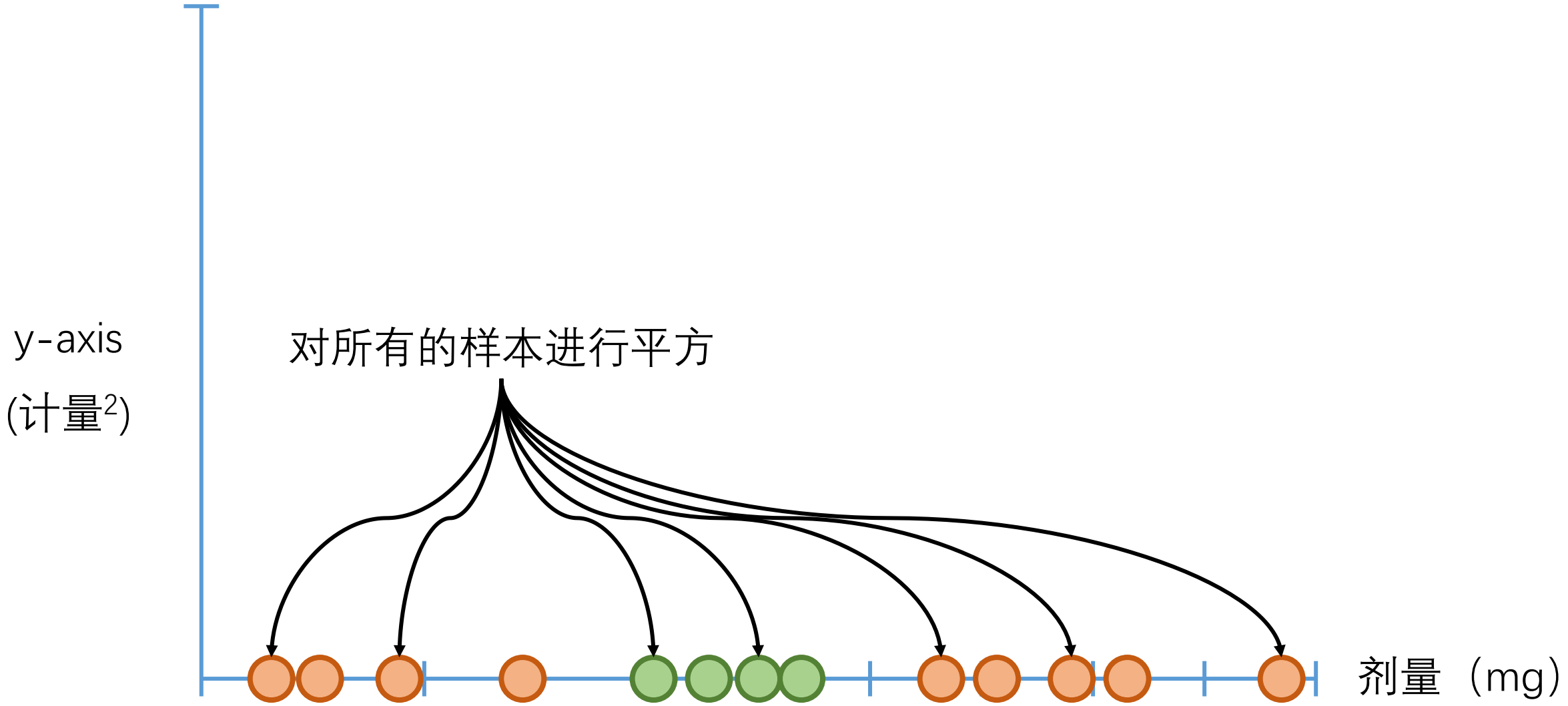
和**支持向量分类器**更好的方法吗？

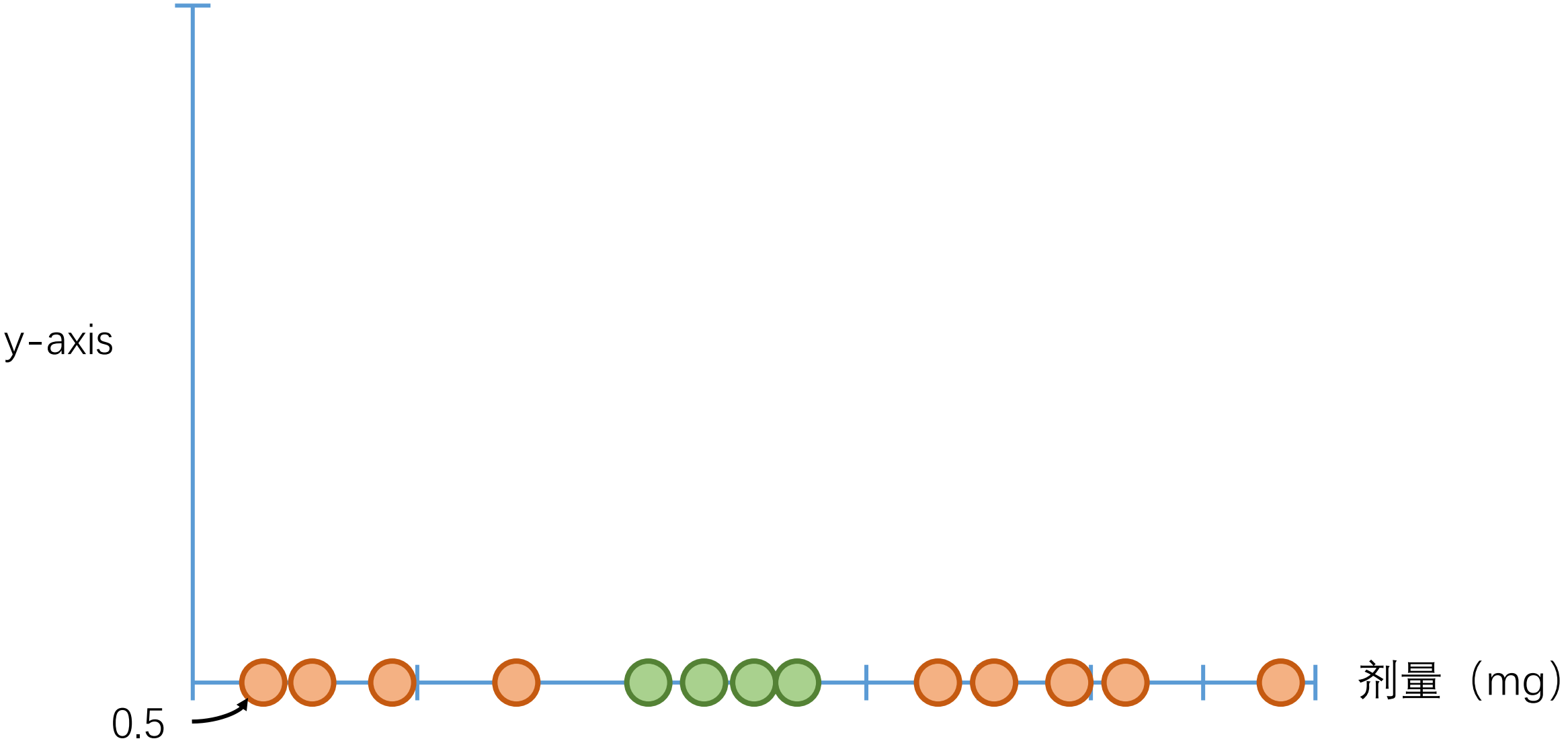


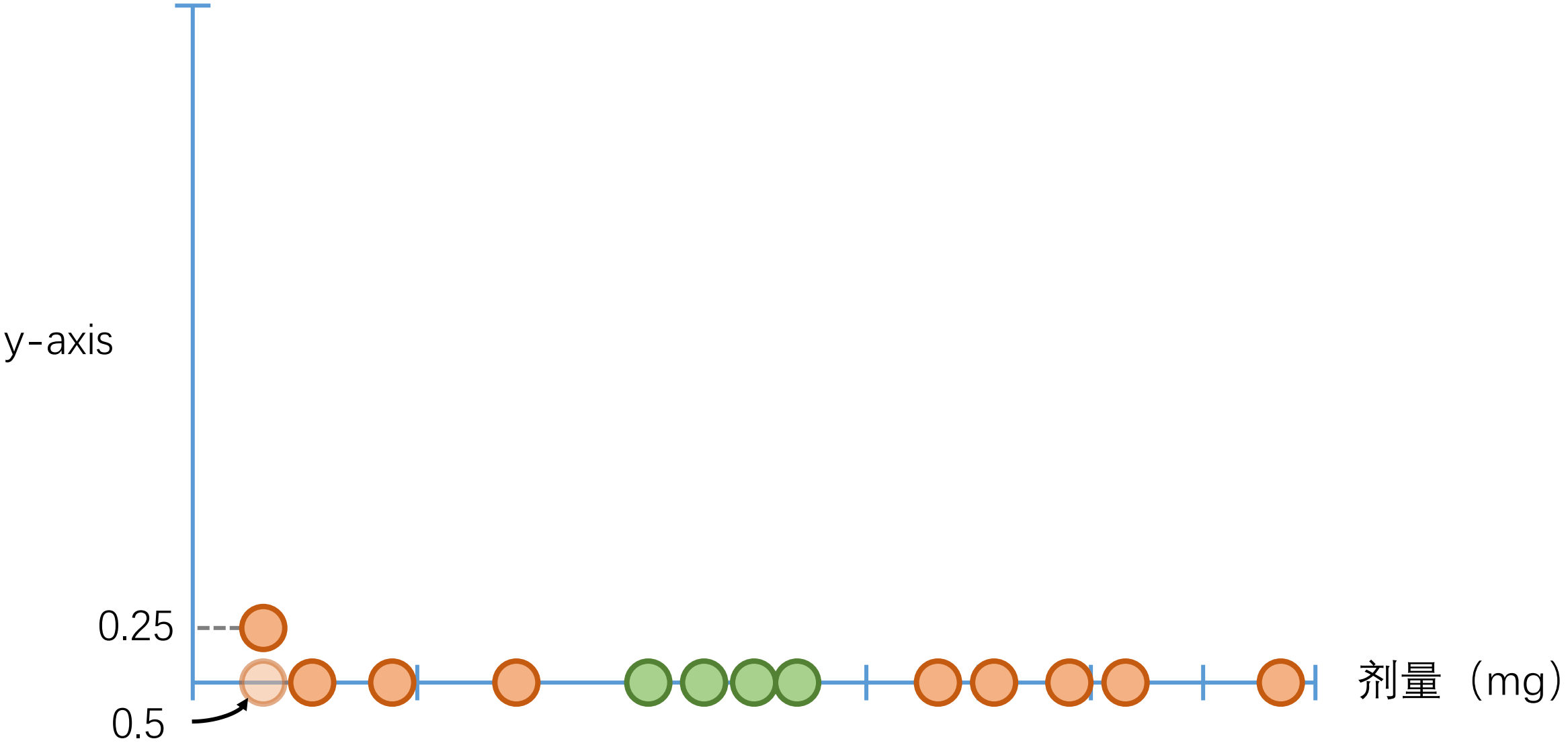
支持向量机

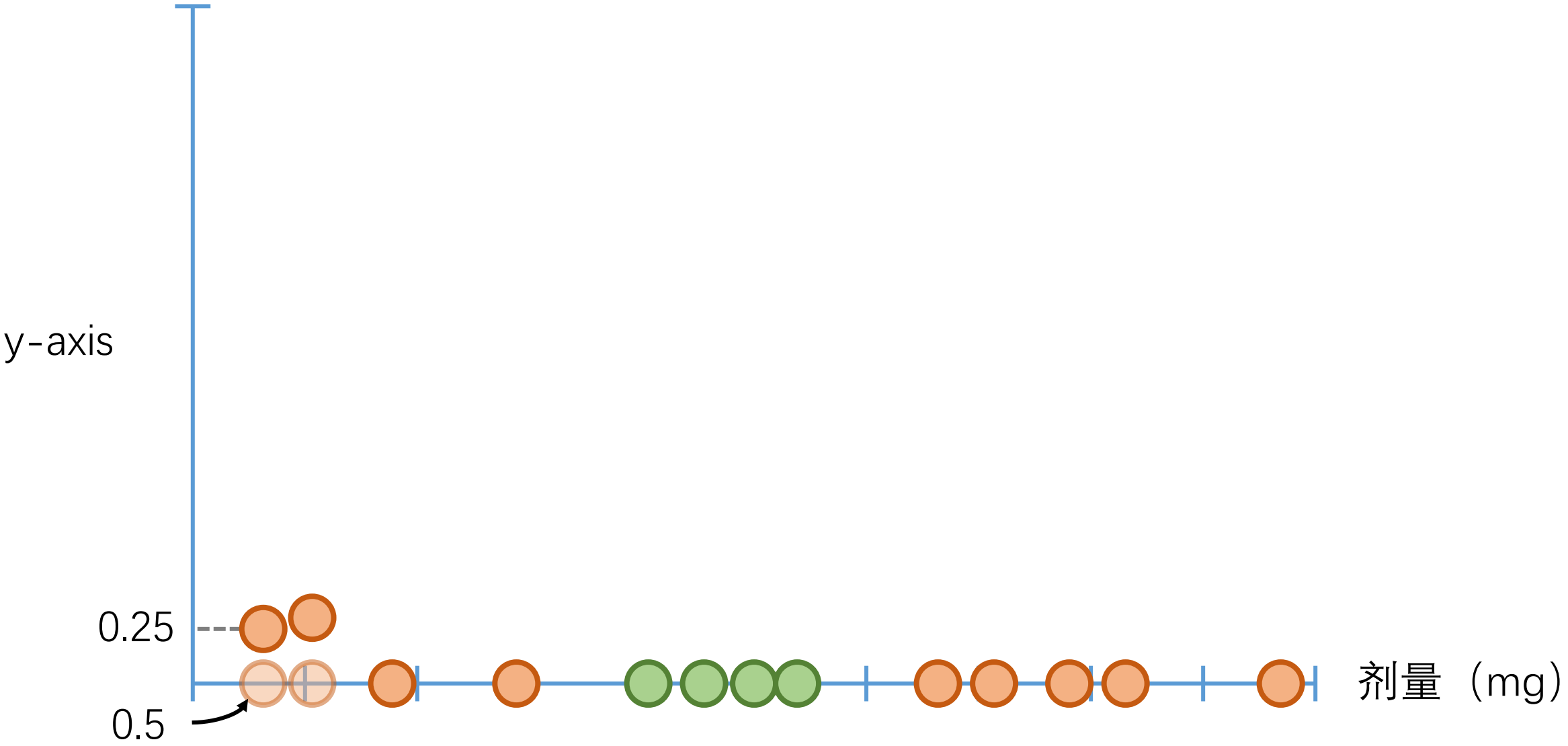
Support Vector Machine

首先增加一个y轴




y-axis
0.5
剂量（mg）

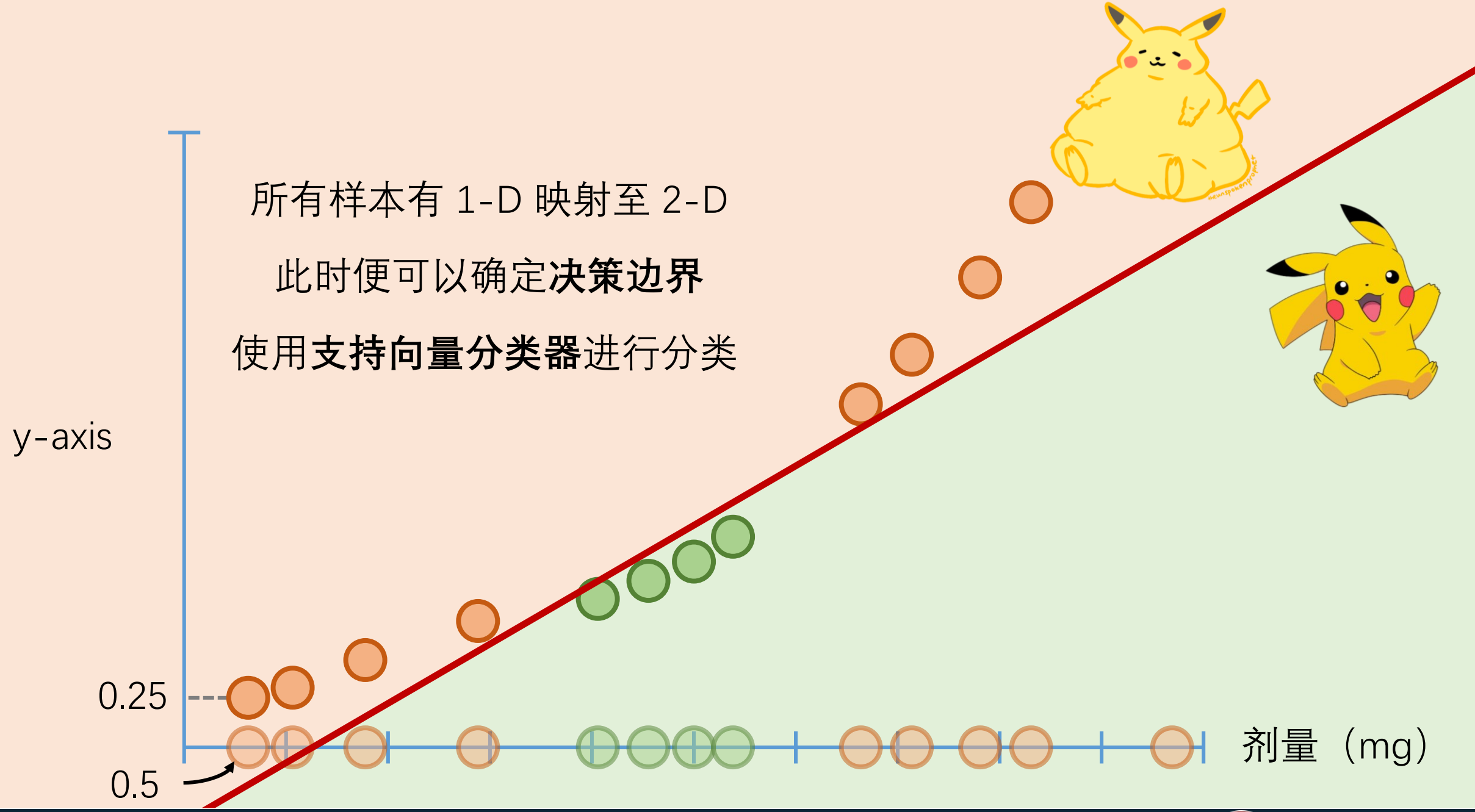

y-axis
0.25
0.5
剂量（mg）


y-axis
0.25
0.5
剂量（mg）

所有样本有 1-D 映射至 2-D

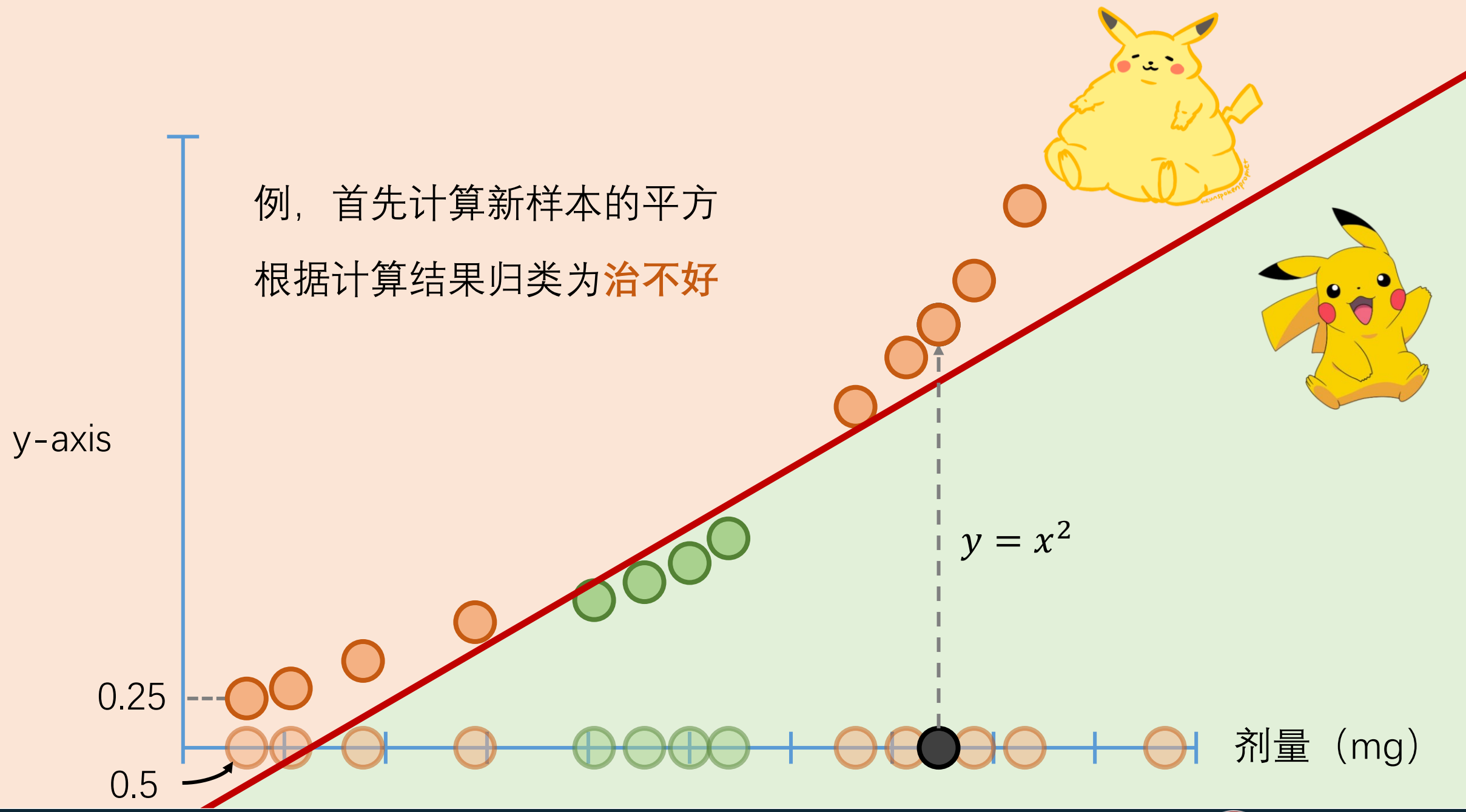
此时便可以确定**决策边界**

使用**支持向量分类器**进行分类



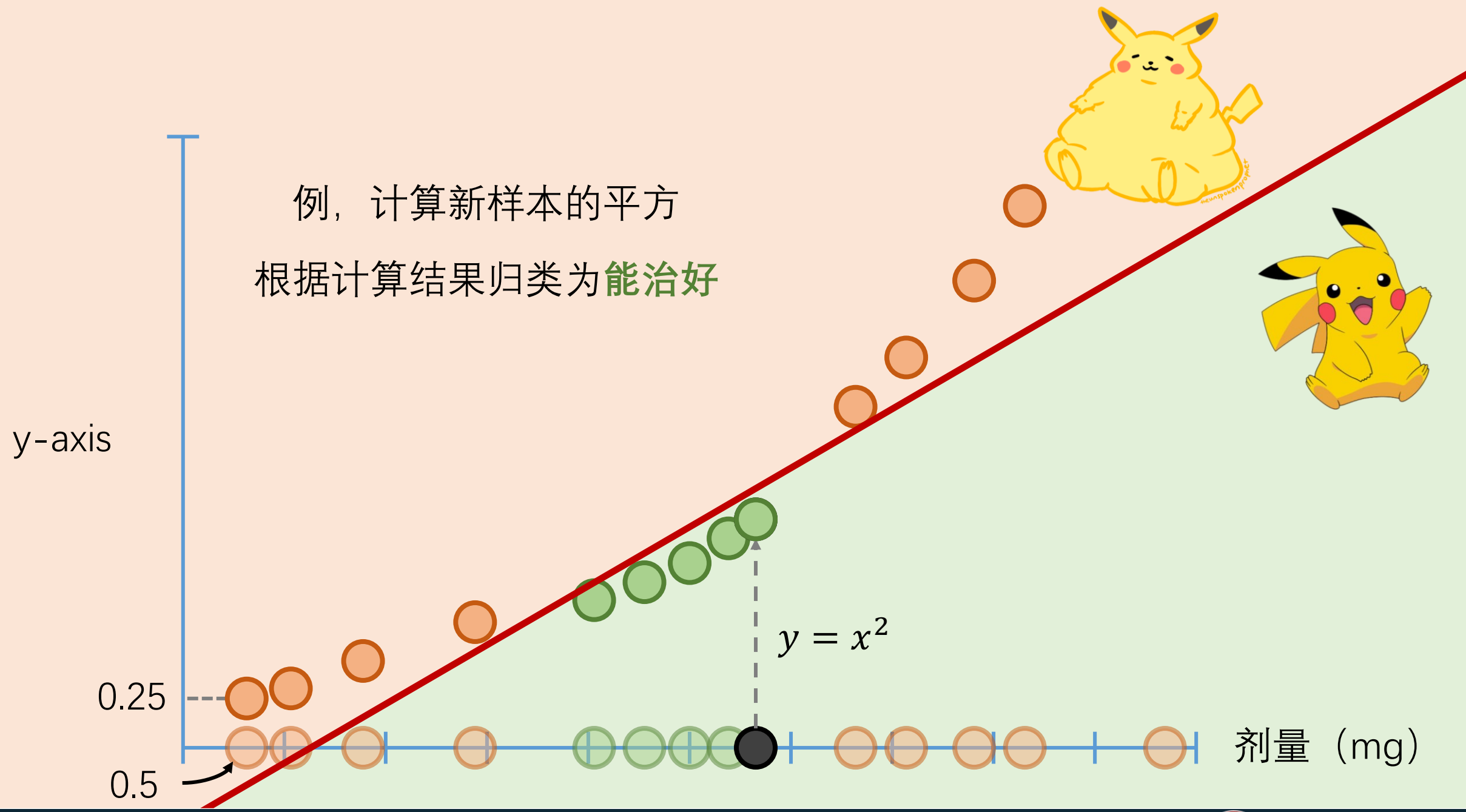
例，首先计算新样本的平方

根据计算结果归类为**治不好**

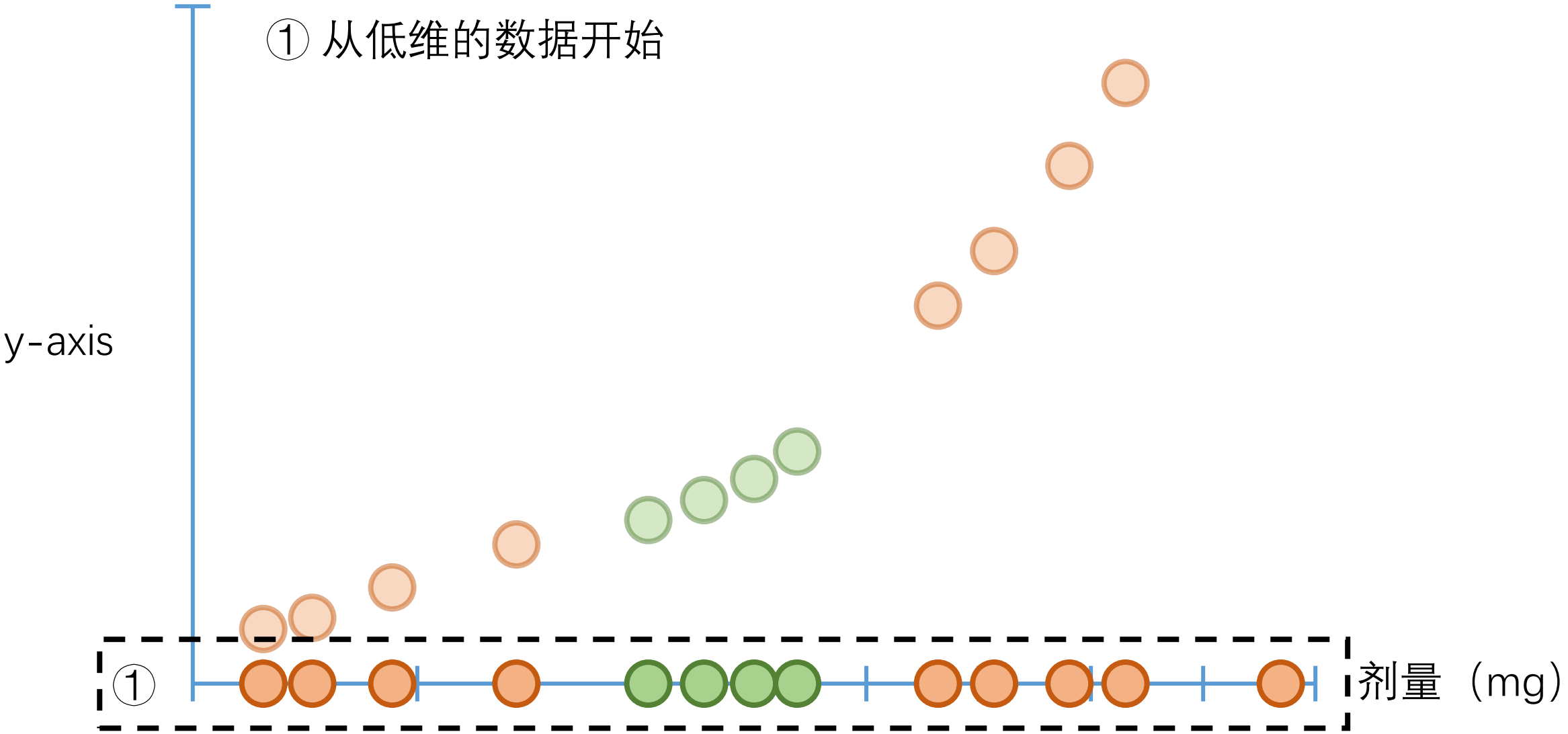


例，计算新样本的平方

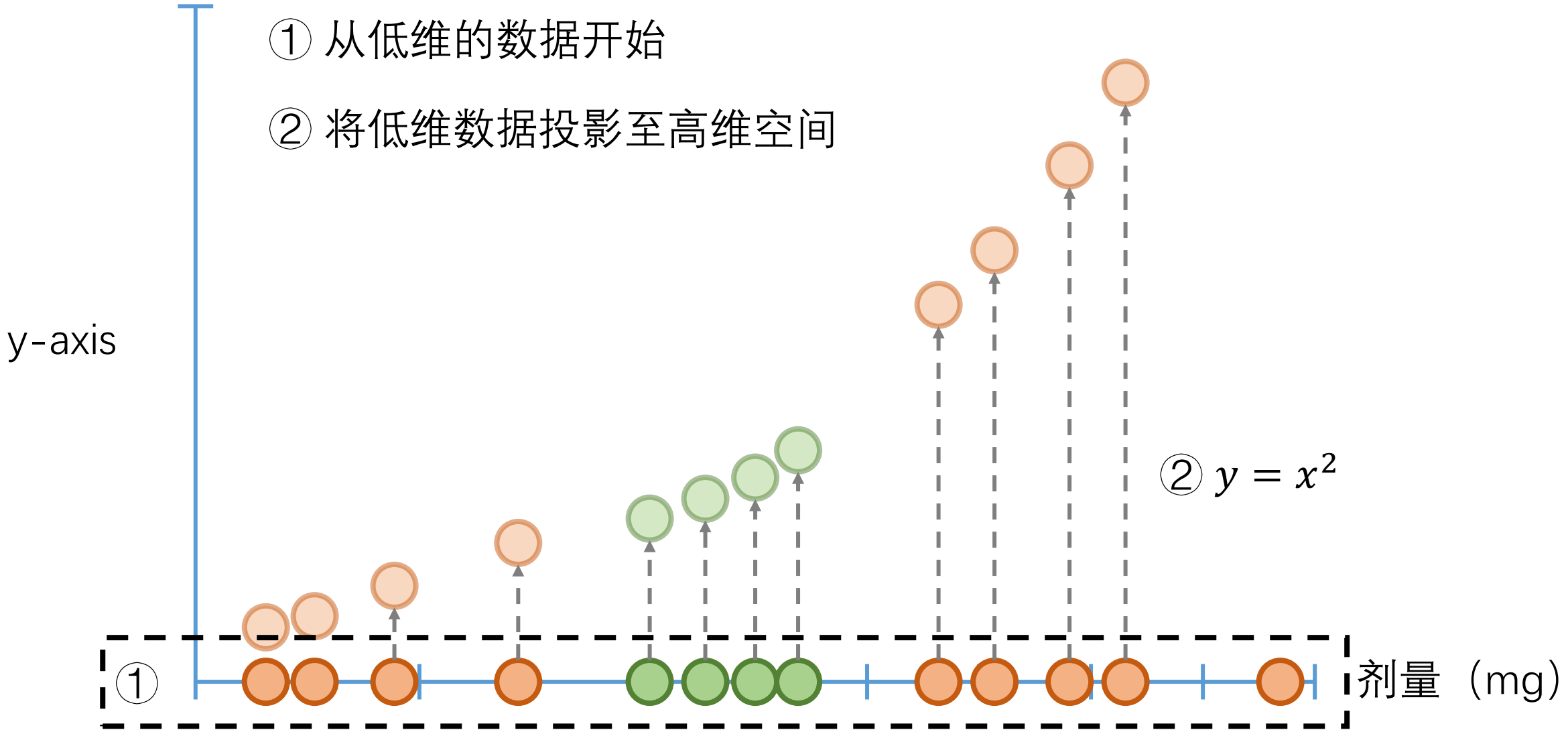
根据计算结果归类为**能治好**



支持向量机的主要思想



支持向量机的主要思想

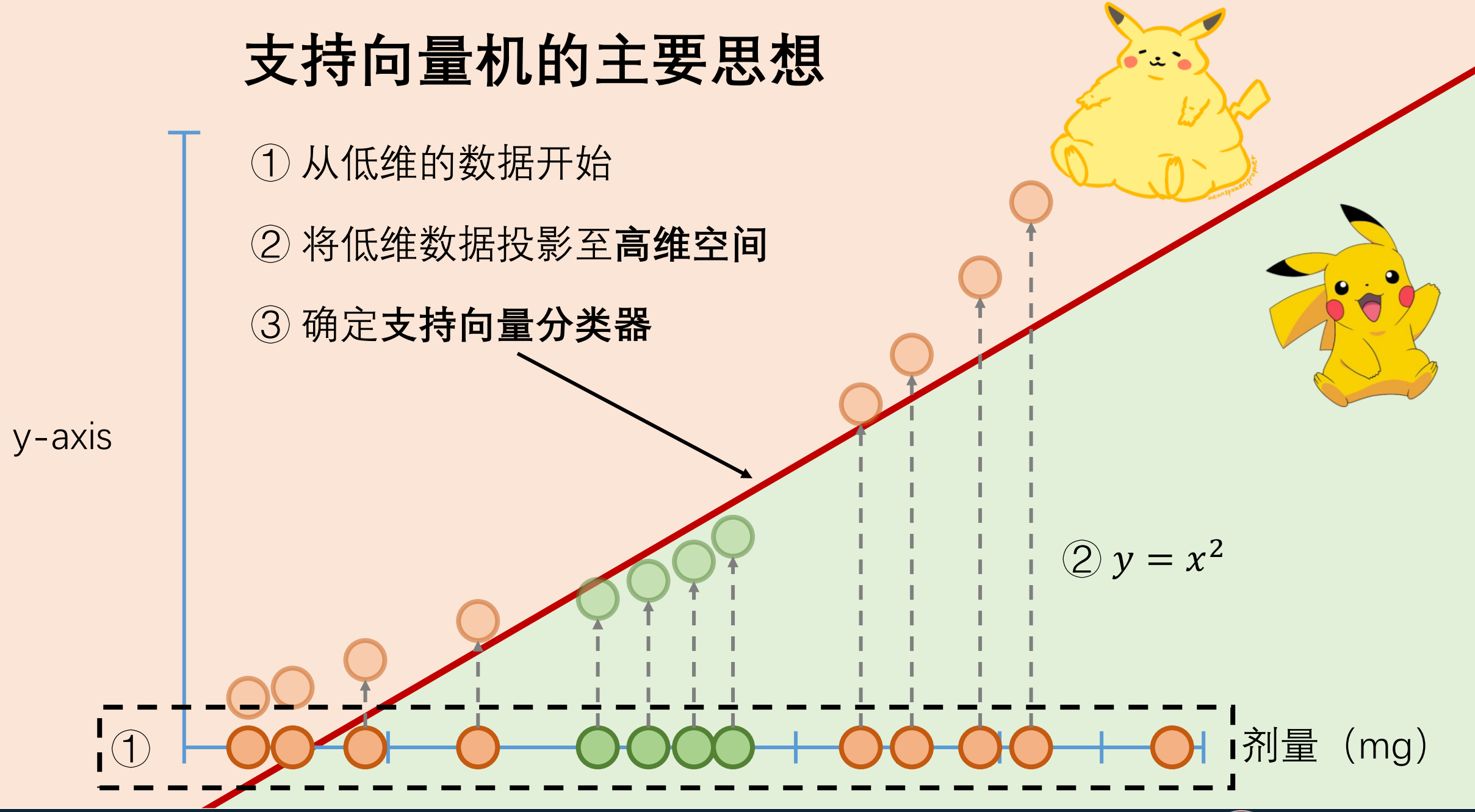


支持向量机的主要思想

① 从低维的数据开始

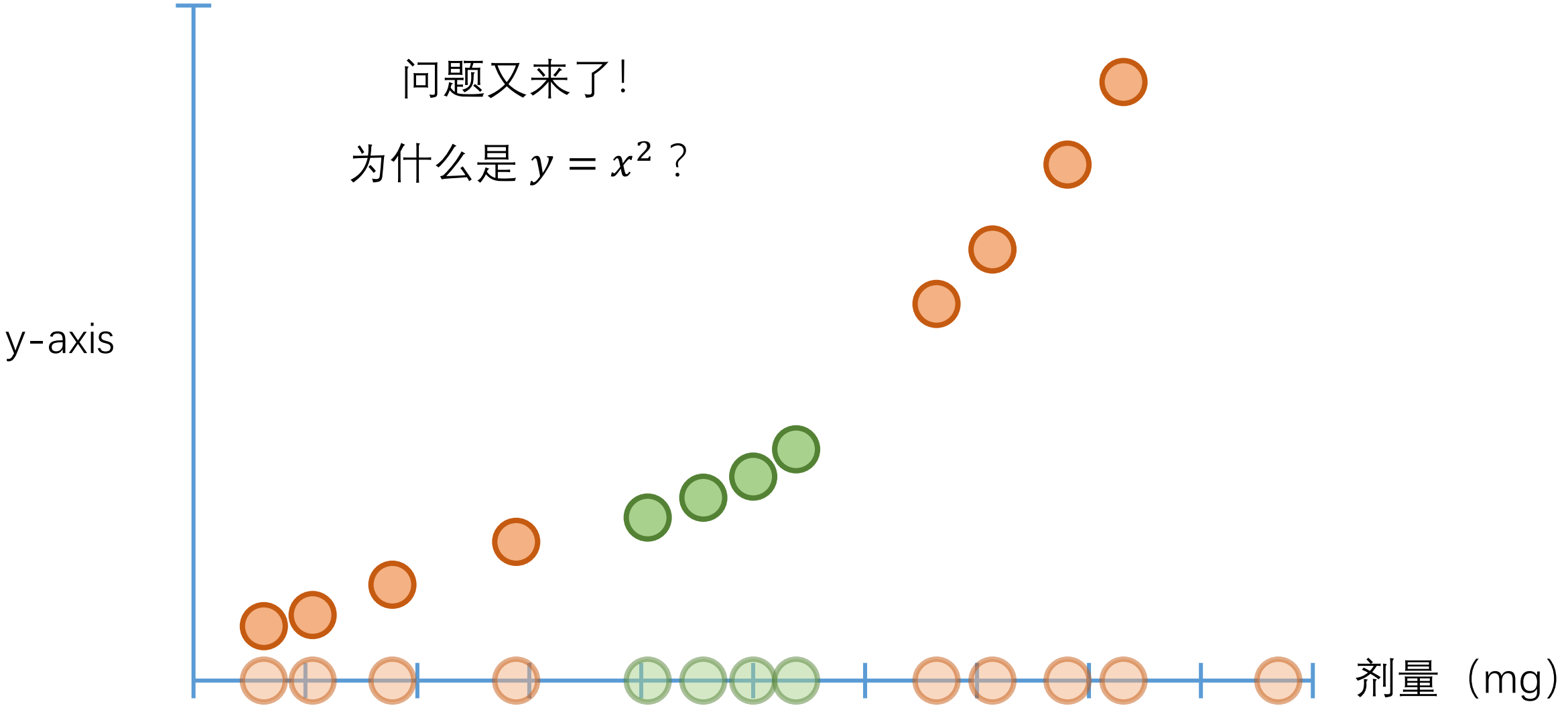
② 将低维数据投影至**高维空间**

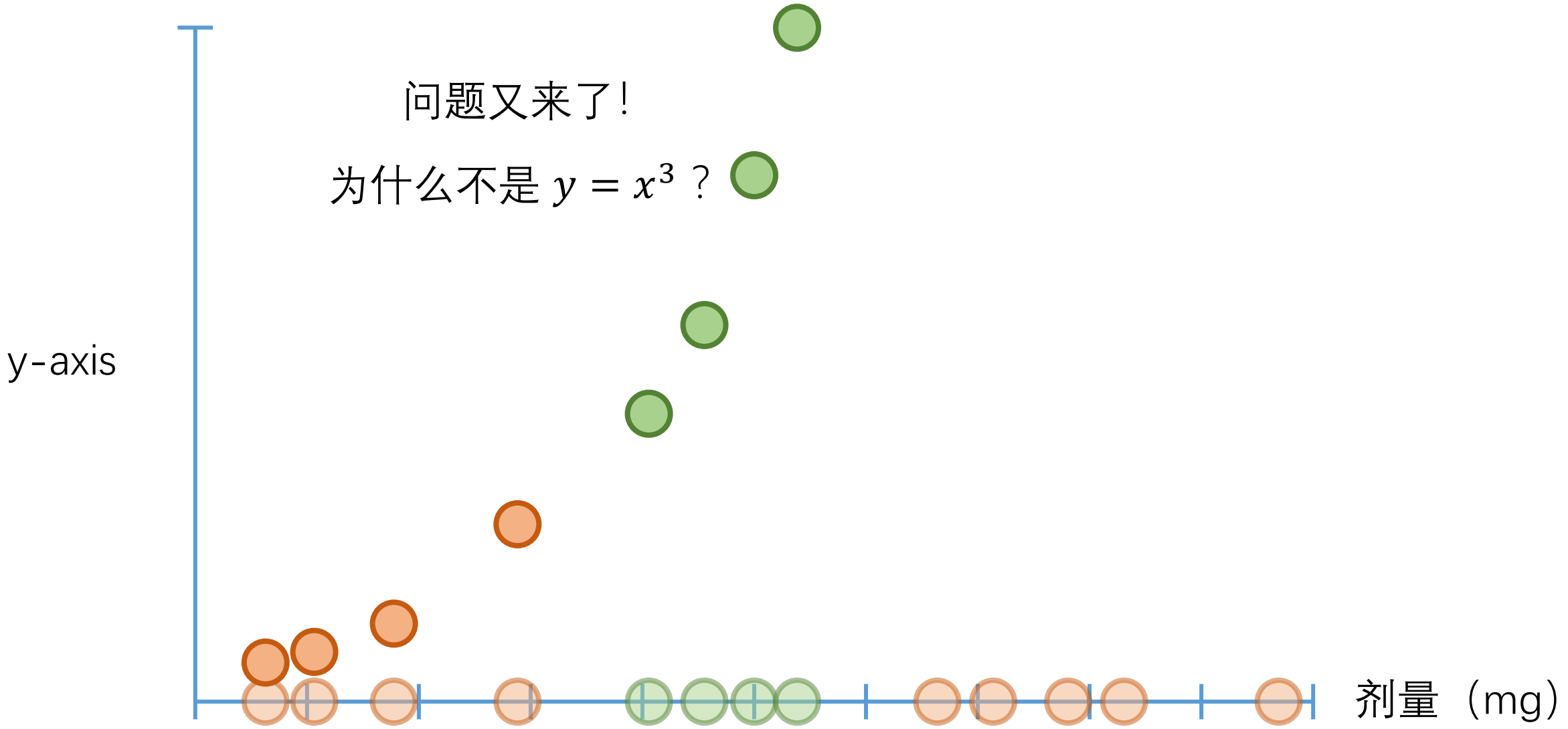
③ 确定**支持向量分类器**

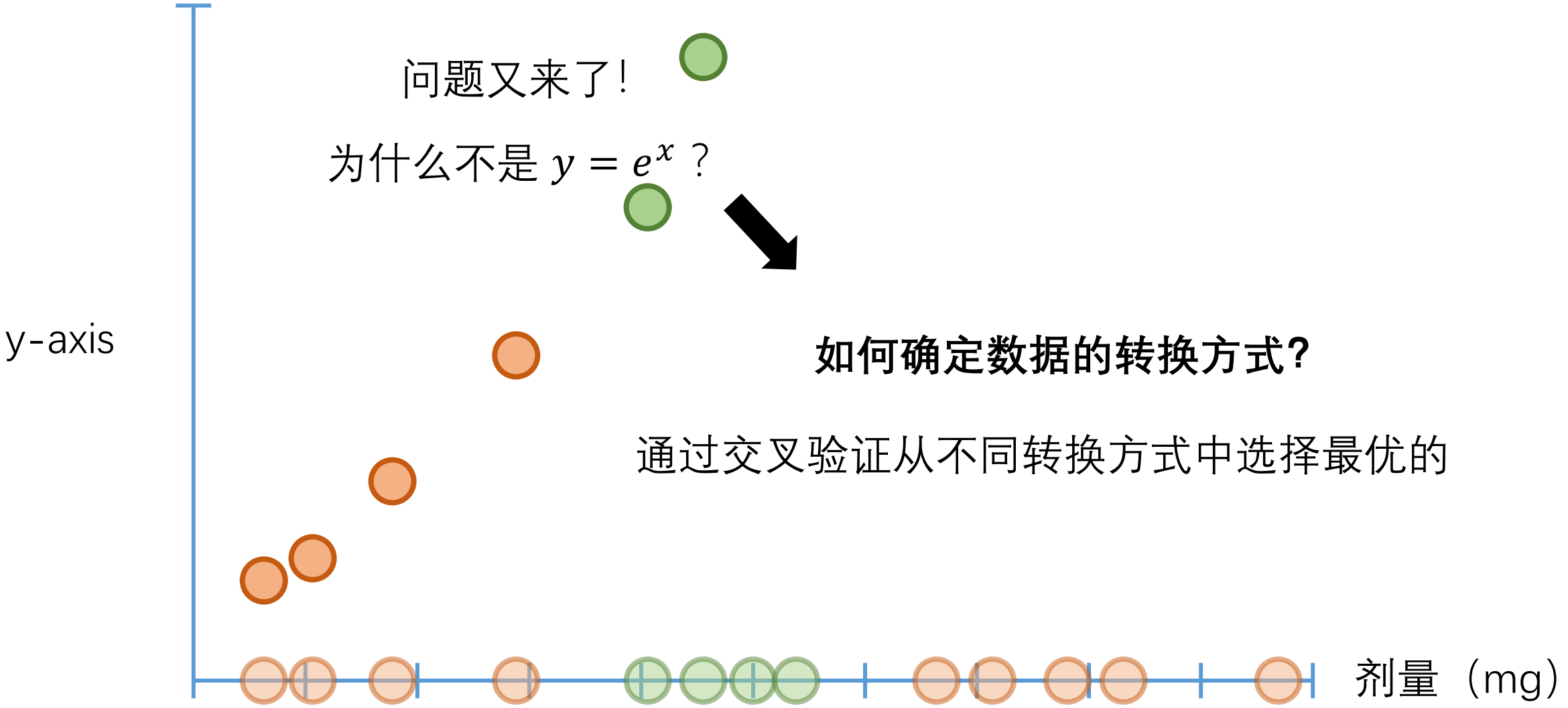


问题又来了！

为什么是 $y = x^2$ ？




问题又来了！
为什么不是 $y = x^3$ ？
y-axis
剂量（mg）



问题又来了！

为什么不是 $y = e^x$ ？

如何确定数据的转换方式?

通过交叉验证从不同转换方式中选择最优的

一个非常实际的问题——如何降低计算量?

当 $\boldsymbol{x}$ 是 3-D 向量时，令 $\boldsymbol{x} = (a, b, c)$

有 $\boldsymbol{x}^2$ 的维度为 6，$\boldsymbol{x}^2 = (a^2, b^2, c^2, 2ab, 2ac, 2bc)$

有 $\boldsymbol{x}^3$ 的维度为 10，$\boldsymbol{x}^3 = (a^3, b^3, c^3, a^2b, a^2c, ab^2, b^2c, ac^2, bc^2, abc)$

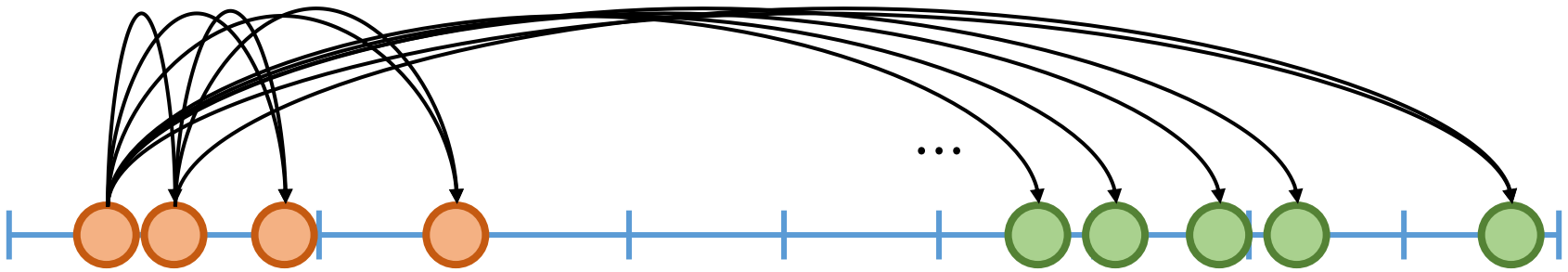
随着阶数增加，高维特征向量所需的存储空间和计算耗时成倍增长

解决这个非常实际的问题——核函数 Kernel Function

在不生成高维特征向量的情况向，计算样本在高维空间中的距离

以**多项式核函数（Polynomial Kernel）**为例，d 表示阶数

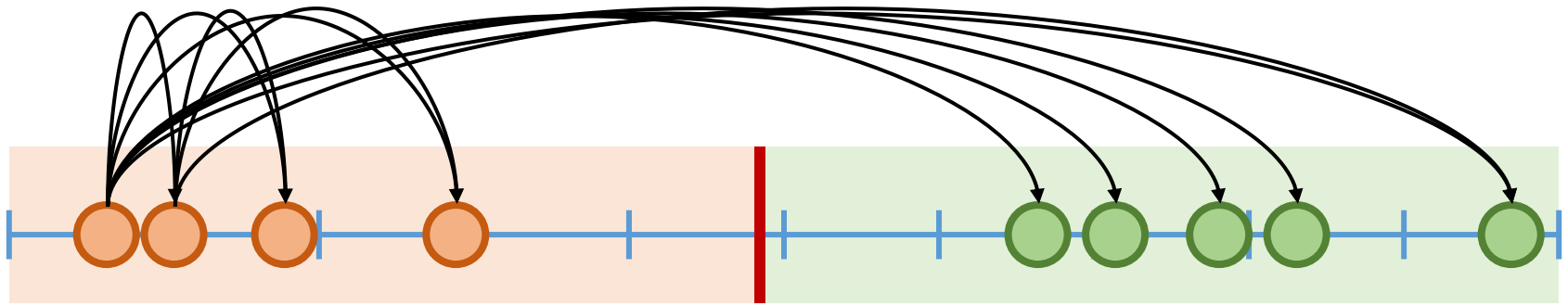
当 $d = 1$ 时：逐对计算向量间的距离



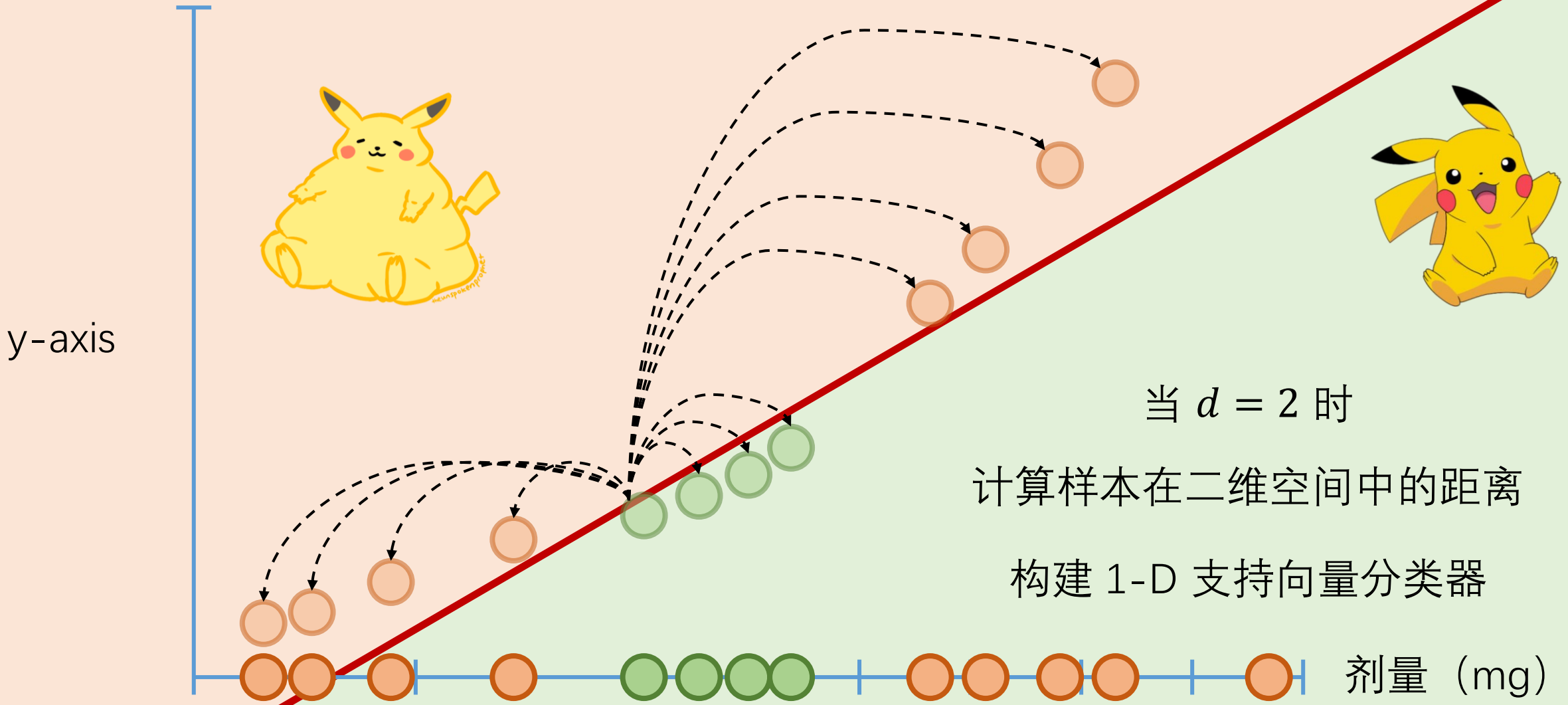
解决这个非常实际的问题——核函数 Kernel Function

根据这些距离，找出合适的**软分类间隔**和**决策边界**

构建 0-D **支持向量分类器**



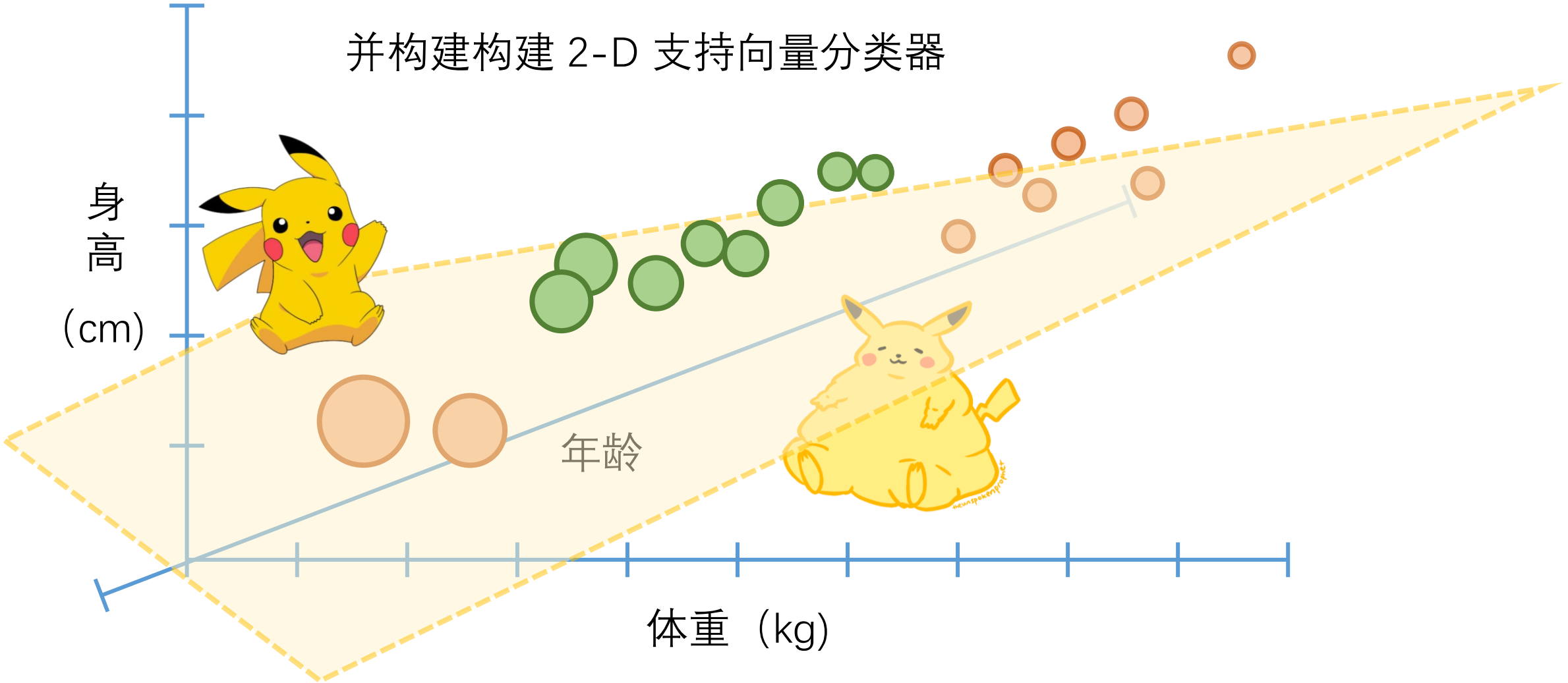
解决这个非常实际的问题——核函数 Kernel Function



解决这个非常实际的问题——核函数 Kernel Function

当 $d = 3$ 时，计算三维空间中的距离，

并构建构建 2-D 支持向量分类器



解决这个非常实际的问题——核函数 Kernel Function

以此类推至更高维……

同时使用交叉验证选出最佳的 d 值

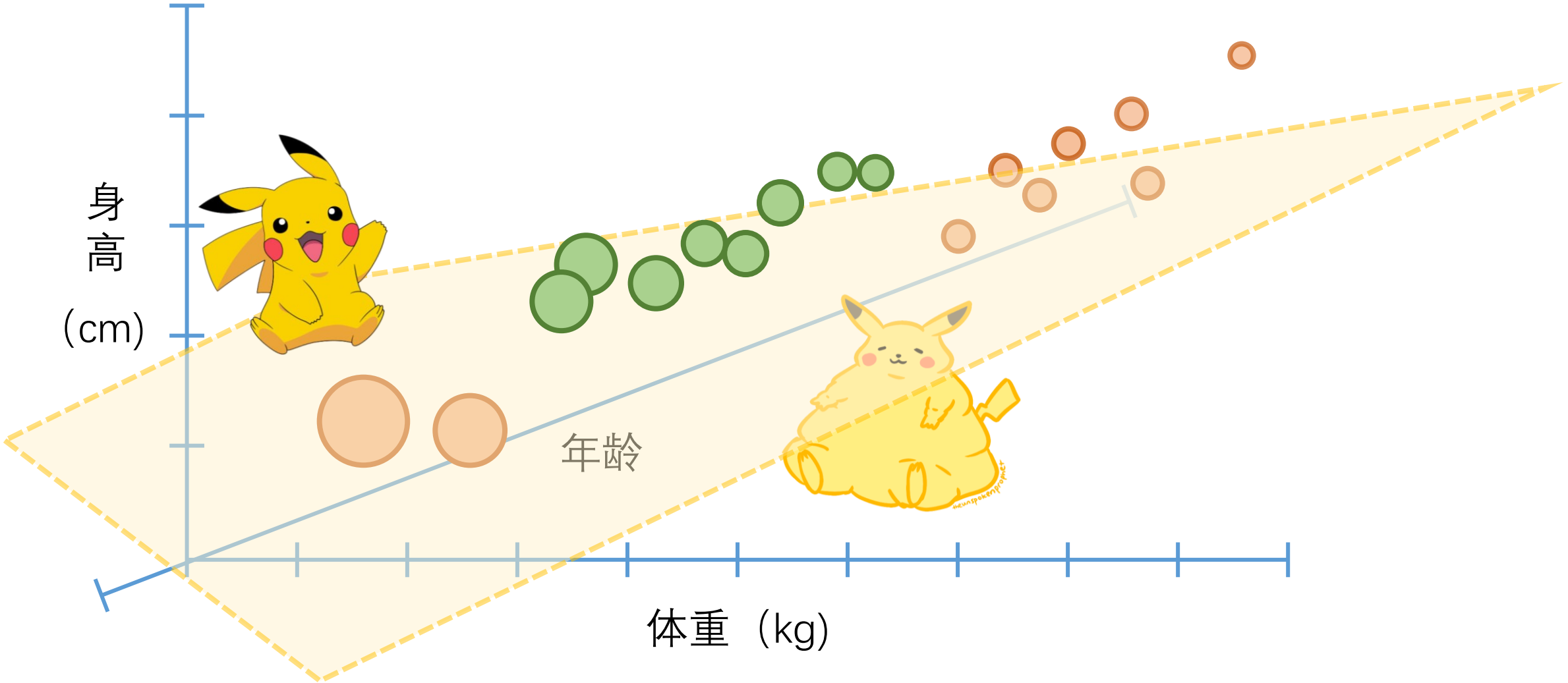
【再次强调】

核函数仅计算样本在高维空间中的距离

<u>不会将原向量转换至高维空间</u>

支持向量机通过超平面进行二分类

如何解决多分类问题呢?



SVM的多分类——一对多方法 One-vs-Rest

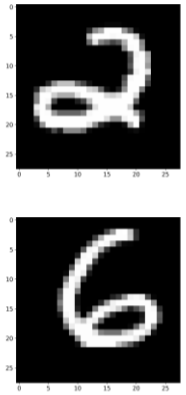
SVM0: 0 vs Rest

SVM1: 1 vs Rest

SVM1: 2 vs Rest

⋮

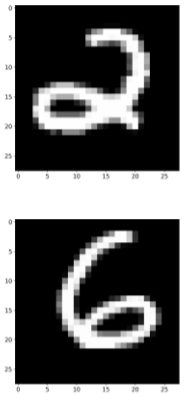
SVM9: 9 vs Rest



SVMs

	0	1	2	3	4	5	6	7	8	9
SVM0: 0 vs Rest	1	0	0	0	0	0	0	0	0	0
SVM1: 1 vs Rest	0	1	0	0	0	0	0	0	0	0
SVM1: 2 vs Rest	0	0	1	0	0	0	0	0	0	0
SVM9: 9 vs Rest	0	0	0	0	0	0	0	0	0	1
	0.02	0.04	57.7	7.75	0.00	34.0	0.03	0.00	0.43	0.00
	0.03	0.09	3.55	0.02	0.00	0.20	95.4	0.00	0.74	0.00

SVM的多分类——一对一方法 One-vs-One



SVMs

0	1	2	3	4	5	6	7	8	9
	SVM 0-1	SVM 0-2	SVM 0-3	SVM 0-4	SVM 0-5	SVM 0-6	SVM 0-7	SVM 0-8	SVM 0-9
SVM 1-0		SVM 1-2	SVM 1-3	SVM 1-4	SVM 1-5	SVM 1-6	SVM 1-7	SVM 1-8	SVM 1-9
⋮									
SVM 9-0	SVM 9-1	SVM 9-2	SVM 9-3	SVM 9-4	SVM 9-5	SVM 9-6	SVM 9-7	SVM 9-8	

Vote

7	7	17	11	6	7	8	10	7	1
5	8	6	5	10	12	18	5	7	5

小结

肥胖？正常？

最大边际分类器 ------ 对异常较为敏感

↓

升维至线性可分 ← 无法解决线性不可分 ---- 支持向量分类器

↓

支持向量机 Support Vector Machine

将线性不可分的数据投影至高维特征空间，直至找到一个超平面，即能够正确分类的支持向量分类器

↓

核函数之一：多项式核函数

↓

二分类到多分类：One-vs-Rest, One-vs-One